

claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_d251ad6c-472\agent-acaceea07ce3cf7ff.jsonl`
- SHA-256: `8b3ae751e975d328b9db88b0e3fadadf3c45ef68b5be5570faeec79b81992a8c`
- Source modified: `2026-06-08T08:40:11+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_d251ad6c-472`
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`
```

Transcript

1. user (2026-06-08T08:39:01.816Z)

Read the collector DB/provenance + observability layer.

Files: C:/Users/avidu/Projects/sports-data-collector/src/core/db.py,
C:/Users/avidu/Projects/sports-data-collector/src/reporting/adapters.py,
C:/Users/avidu/Projects/sports-data-collector/src/reporting/spec.yaml,
C:/Users/avidu/Projects/sports-data-collector/src/reporting/__init__.py,
C:/Users/avidu/Projects/sports-data-collector/tests/test_local_import.py,
C:/Users/avidu/Projects/sports-data-collector/tests/test_validator.py.

Determine, with anchors:

- How an ingested annotation set is registered/persisted with provenance (source, license, sport, video id) and what status/maturity fields (if any) exist.
- How import emits an observability report (what flags it raises for local imports).
- How license/provenance is tracked (Proprietary self-rated vs vendor-delivered).
- Gaps for: tracking manual self-rated labels, partial-label maturity (needs_review -> confirmed -> gold), and label-quality/regression over re-tagging.

2. assistant (2026-06-08T08:39:03.784Z)

I'll analyze the collector DB/provenance and observability layer across the specified files. Let me start by reading them systematically.

3. user (2026-06-08T08:39:04.048Z)

```
1      import sqlite3
2      import os
3      from contextlib import contextmanager
4      from typing import List, Dict, Any, Optional
5      from src.core.schemas.base import VideoMetadata, SportType, CurationStatus, LicenseType
6      from src.core.schemas.badminton import BadmintonRally
7      from src.core.schemas.tennis import TennisPoint
8      from src.core.schemas.cricket import CricketDelivery
9      from src.core.schemas.pickleball import PickleballRally
10     from src.core.schemas.tabletennis import TableTennisRally
11
12     # Maps each sport to its annotation table and the column(s) that order segments
13     # chronologically. All child tables expose (start_time, end_time), so a
14     # single generic reader can serve every sport ? adding a new sport is one entry
15     # here plus its CREATE TABLE, with no new query code.
16     SEGMENT_TABLES = {
17         SportType.BADMINTON: ("badminton_rallies", "rally_number"),
18         SportType.TENNIS: ("tennis_points", "point_number"),
19         SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
20         SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
21         SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
22     }
```

```

23
24
25 class SportsDatabase:
26     def __init__(self, db_path: str):
27         self.db_path = db_path
28         # Ensure target directory exists
29         os.makedirs(os.path.dirname(os.path.abspath(db_path)), exist_ok=True)
30         self._init_db()
31
32     @contextmanager
33     def _get_connection(self):
34         """Yield a SQLite connection and always close it.
35
36         Used as ``with self._get_connection() as conn:``. A plain
37         ``sqlite3.connect(...)`` used as a context manager commits/rolls back the
38         transaction but does NOT close the connection, which leaks handles and
39         raises ResourceWarnings. This wrapper guarantees the connection is closed
40         on exit; writers still commit explicitly inside the block.
41         """
42         conn = sqlite3.connect(self.db_path)
43         conn.row_factory = sqlite3.Row
44         # SQLite disables foreign keys per-connection by default; enable so the
45         # ON DELETE CASCADE declarations on the child tables actually fire.
46         conn.execute("PRAGMA foreign_keys = ON")
47         try:
48             yield conn
49         finally:
50             conn.close()
51
52     def _init_db(self):
53         with self._get_connection() as conn:
54             cursor = conn.cursor()
55
56             # 1. Create common videos table
57             cursor.execute("""
58             CREATE TABLE IF NOT EXISTS videos (
59                 video_id TEXT PRIMARY KEY,
60                 sport TEXT NOT NULL,
61                 video_url TEXT,
62                 local_path TEXT,
63                 license_type TEXT NOT NULL,
64                 license_url TEXT,
65                 is_commercial INTEGER NOT NULL DEFAULT 0,
66                 status TEXT NOT NULL DEFAULT 'staging',
67                 match_name TEXT NOT NULL,
68                 fps REAL,
69                 resolution TEXT
70             )
71             """)
72
73             # 2. Create badminton_rallies table
74             cursor.execute("""
75             CREATE TABLE IF NOT EXISTS badminton_rallies (
76                 video_id TEXT NOT NULL,
77                 rally_number INTEGER NOT NULL,
78                 start_time REAL NOT NULL,
79                 end_time REAL NOT NULL,
80                 score_before TEXT,
81                 score_after TEXT,
82                 shots_count INTEGER,
83                 ending_reason TEXT,
84                 last_shot_outcome TEXT,
85                 PRIMARY KEY (video_id, rally_number),
86                 FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
87             )

```

```

88         """)
89
90     # 3. Create tennis_points table
91     cursor.execute("""
92     CREATE TABLE IF NOT EXISTS tennis_points (
93         video_id TEXT NOT NULL,
94         point_number INTEGER NOT NULL,
95         start_time REAL NOT NULL,
96         end_time REAL NOT NULL,
97         set_score TEXT,
98         game_score TEXT,
99         server TEXT,
100        ending_type TEXT,
101        PRIMARY KEY (video_id, point_number),
102        FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
103    )
104    """)
105
106     # 4. Create cricket_deliveries table
107     cursor.execute("""
108     CREATE TABLE IF NOT EXISTS cricket_deliveries (
109         video_id TEXT NOT NULL,
110         over_num INTEGER NOT NULL,
111         ball_num INTEGER NOT NULL,
112         start_time REAL NOT NULL,
113         end_time REAL NOT NULL,
114         batsman TEXT,
115         bowler TEXT,
116         runs INTEGER,
117         wicket INTEGER DEFAULT 0,
118         extras INTEGER DEFAULT 0,
119         PRIMARY KEY (video_id, over_num, ball_num),
120         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
121     )
122     """)
123
124     # 5. Create pickleball_rallies table
125     cursor.execute("""
126     CREATE TABLE IF NOT EXISTS pickleball_rallies (
127         video_id TEXT NOT NULL,
128         rally_number INTEGER NOT NULL,
129         start_time REAL NOT NULL,
130         end_time REAL NOT NULL,
131         score_before TEXT,
132         score_after TEXT,
133         shots_count INTEGER,
134         ending_reason TEXT,
135         last_shot_outcome TEXT,
136         PRIMARY KEY (video_id, rally_number),
137         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
138     )
139     """)
140
141     # 6. Create tabletennis_rallies table
142     cursor.execute("""
143     CREATE TABLE IF NOT EXISTS tabletennis_rallies (
144         video_id TEXT NOT NULL,
145         rally_number INTEGER NOT NULL,
146         start_time REAL NOT NULL,
147         end_time REAL NOT NULL,
148         score_before TEXT,
149         score_after TEXT,
150         shots_count INTEGER,
151         ending_reason TEXT,
152         last_shot_outcome TEXT,

```

```

153         PRIMARY KEY (video_id, rally_number),
154         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
155     )
156     """)
157
158     # Migration: add last_shot_outcome to rally tables that predate the
159     # column. CREATE TABLE IF NOT EXISTS won't alter an existing table, so
160     # do it explicitly and idempotently (skip if the column is already there).
161     for tbl in ("badminton_rallies", "pickleball_rallies",
162 "tabletennis_rallies"):
163         existing = [r[1] for r in cursor.execute(f"PRAGMA
164 table_info({tbl})").fetchall()]
165         if "last_shot_outcome" not in existing:
166             cursor.execute(f"ALTER TABLE {tbl} ADD COLUMN last_shot_outcome
167 TEXT")
168
169         conn.commit()
170
171 def insert_video(self, video: VideoMetadata) -> bool:
172     """Insert a video entry, or update it in place if it already exists.
173
174     Uses an UPSERT rather than INSERT OR REPLACE: REPLACE would DELETE the
175     existing row, which (with foreign keys enabled) would cascade-delete the
176     video's child annotations. UPSERT mutates the row in place and leaves
177     children untouched.
178     """
179     with self._get_connection() as conn:
180         cursor = conn.cursor()
181         cursor.execute("""
182             INSERT INTO videos (
183                 video_id, sport, video_url, local_path, license_type,
184                 license_url, is_commercial, status, match_name, fps, resolution
185             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
186             ON CONFLICT(video_id) DO UPDATE SET
187                 sport=excluded.sport,
188                 video_url=excluded.video_url,
189                 local_path=excluded.local_path,
190                 license_type=excluded.license_type,
191                 license_url=excluded.license_url,
192                 is_commercial=excluded.is_commercial,
193                 status=excluded.status,
194                 match_name=excluded.match_name,
195                 fps=excluded.fps,
196                 resolution=excluded.resolution
197             """, (
198                 video.video_id,
199                 video.sport.value,
200                 video.video_url,
201                 video.local_path,
202                 video.license_type.value,
203                 video.license_url,
204                 1 if video.is_commercial else 0,
205                 video.status.value,
206                 video.match_name,
207                 video.fps,
208                 video.resolution
209             ))
210         conn.commit()
211         return True
212
213 def insert_badminton_rallies(self, rallies: List[BadmintonRally]) -> int:
214     if not rallies:
215         return 0
216     with self._get_connection() as conn:
217         cursor = conn.cursor()

```

```

215         # First clear existing rallies for this video_id to avoid key conflicts on
updates
216         video_id = rallies[0].video_id
217         cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
218
219         for rally in rallies:
220             cursor.execute("""
221             INSERT INTO badminton_rallies (
222                 video_id, rally_number, start_time, end_time,
223                 score_before, score_after, shots_count, ending_reason,
last_shot_outcome
224             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
225             """, (
226                 rally.video_id,
227                 rally.rally_number,
228                 rally.start_time,
229                 rally.end_time,
230                 rally.score_before,
231                 rally.score_after,
232                 rally.shots_count,
233                 rally.ending_reason,
234                 rally.last_shot_outcome
235             ))
236         conn.commit()
237         return len(rallies)
238
239     def insert_tennis_points(self, points: List[TennisPoint]) -> int:
240         if not points:
241             return 0
242         with self._get_connection() as conn:
243             cursor = conn.cursor()
244             video_id = points[0].video_id
245             cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
246
247             for pt in points:
248                 cursor.execute("""
249                 INSERT INTO tennis_points (
250                     video_id, point_number, start_time, end_time,
251                     set_score, game_score, server, ending_type
252                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
253                 """, (
254                     pt.video_id,
255                     pt.point_number,
256                     pt.start_time,
257                     pt.end_time,
258                     pt.set_score,
259                     pt.game_score,
260                     pt.server,
261                     pt.ending_type
262                 ))
263             conn.commit()
264             return len(points)
265
266     def insert_cricket_deliveries(self, deliveries: List[CricketDelivery]) -> int:
267         if not deliveries:
268             return 0
269         with self._get_connection() as conn:
270             cursor = conn.cursor()
271             video_id = deliveries[0].video_id
272             cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))
273
274             for deliv in deliveries:
275                 cursor.execute("""

```

```

276         INSERT INTO cricket_deliveries (
277             video_id, over_num, ball_num, start_time, end_time,
278             batsman, bowler, runs, wicket, extras
279         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
280         """ , (
281             deliv.video_id,
282             deliv.over,
283             deliv.ball,
284             deliv.start_time,
285             deliv.end_time,
286             deliv.batsman,
287             deliv.bowler,
288             deliv.runs,
289             1 if deliv.wicket else 0,
290             deliv.extras
291         ))
292         conn.commit()
293         return len(deliveries)
294
295     def insert_pickleball_rallies(self, rallies: List[PickleballRally]) -> int:
296         if not rallies:
297             return 0
298         with self._get_connection() as conn:
299             cursor = conn.cursor()
300             video_id = rallies[0].video_id
301             cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
302 (video_id,))
303
304             for rally in rallies:
305                 cursor.execute("""
306                     INSERT INTO pickleball_rallies (
307                         video_id, rally_number, start_time, end_time,
308                         score_before, score_after, shots_count, ending_reason,
309                         last_shot_outcome
310                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
311                     """ , (
312                         rally.video_id,
313                         rally.rally_number,
314                         rally.start_time,
315                         rally.end_time,
316                         rally.score_before,
317                         rally.score_after,
318                         rally.shots_count,
319                         rally.ending_reason,
320                         rally.last_shot_outcome
321                     ))
322                 conn.commit()
323                 return len(rallies)
324
325     def insert_tabletennis_rallies(self, rallies: List[TableTennisRally]) -> int:
326         if not rallies:
327             return 0
328         with self._get_connection() as conn:
329             cursor = conn.cursor()
330             video_id = rallies[0].video_id
331             cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
332 (video_id,))
333
334             for rally in rallies:
335                 cursor.execute("""
336                     INSERT INTO tabletennis_rallies (
337                         video_id, rally_number, start_time, end_time,
338                         score_before, score_after, shots_count, ending_reason,
339                         last_shot_outcome
340                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

```

```

337         """", (
338             rally.video_id,
339             rally.rally_number,
340             rally.start_time,
341             rally.end_time,
342             rally.score_before,
343             rally.score_after,
344             rally.shots_count,
345             rally.ending_reason,
346             rally.last_shot_outcome
347         ))
348     conn.commit()
349     return len(rallies)
350
351 def get_video(self, video_id: str) -> Optional[VideoMetadata]:
352     with self._get_connection() as conn:
353         cursor = conn.cursor()
354         cursor.execute("SELECT * FROM videos WHERE video_id = ?", (video_id,))
355         row = cursor.fetchone()
356         if row:
357             return VideoMetadata(
358                 video_id=row['video_id'],
359                 sport=SportType(row['sport']),
360                 video_url=row['video_url'],
361                 local_path=row['local_path'],
362                 license_type=LicenseType(row['license_type']),
363                 license_url=row['license_url'],
364                 is_commercial=bool(row['is_commercial']),
365                 status=CurationStatus(row['status']),
366                 match_name=row['match_name'],
367                 fps=row['fps'],
368                 resolution=row['resolution']
369             )
370         return None
371
372 def get_videos_by_status(self, status: CurationStatus, sport: Optional[SportType] =
None) -> List[VideoMetadata]:
373     query = "SELECT * FROM videos WHERE status = ?"
374     params = [status.value]
375     if sport:
376         query += " AND sport = ?"
377         params.append(sport.value)
378
379     videos = []
380     with self._get_connection() as conn:
381         cursor = conn.cursor()
382         cursor.execute(query, params)
383         for row in cursor.fetchall():
384             videos.append(VideoMetadata(
385                 video_id=row['video_id'],
386                 sport=SportType(row['sport']),
387                 video_url=row['video_url'],
388                 local_path=row['local_path'],
389                 license_type=LicenseType(row['license_type']),
390                 license_url=row['license_url'],
391                 is_commercial=bool(row['is_commercial']),
392                 status=CurationStatus(row['status']),
393                 match_name=row['match_name'],
394                 fps=row['fps'],
395                 resolution=row['resolution']
396             ))
397     return videos
398
399 def get_status_counts(self) -> List[tuple]:
400     """Return (sport, status, count) rows for the status summary.

```

```

401
402     Public read API so callers (e.g. the CLI `status` command) don't have to
403     reach into the private `_get_connection` context manager.
404     """
405     with self._get_connection() as conn:
406         cursor = conn.cursor()
407         cursor.execute(
408             "SELECT sport, status, COUNT(*) as cnt FROM videos GROUP BY status,
sport"
409         )
410         return [(r["sport"], r["status"], r["cnt"]) for r in cursor.fetchall()]
411
412     def get_badminton_rallies(self, video_id: str) -> List[BadmintonRally]:
413         rallies = []
414         with self._get_connection() as conn:
415             cursor = conn.cursor()
416             cursor.execute("""
417                 SELECT * FROM badminton_rallies
418                 WHERE video_id = ?
419                 ORDER BY rally_number
420                 """, (video_id,))
421             for row in cursor.fetchall():
422                 rallies.append(BadmintonRally(
423                     video_id=row['video_id'],
424                     rally_number=row['rally_number'],
425                     start_time=row['start_time'],
426                     end_time=row['end_time'],
427                     score_before=row['score_before'],
428                     score_after=row['score_after'],
429                     shots_count=row['shots_count'],
430                     ending_reason=row['ending_reason'],
431                     last_shot_outcome=row['last_shot_outcome']
432                 ))
433         return rallies
434
435     def get_tennis_points(self, video_id: str) -> List[TennisPoint]:
436         points = []
437         with self._get_connection() as conn:
438             cursor = conn.cursor()
439             cursor.execute("""
440                 SELECT * FROM tennis_points
441                 WHERE video_id = ?
442                 ORDER BY point_number
443                 """, (video_id,))
444             for row in cursor.fetchall():
445                 points.append(TennisPoint(
446                     video_id=row['video_id'],
447                     point_number=row['point_number'],
448                     start_time=row['start_time'],
449                     end_time=row['end_time'],
450                     set_score=row['set_score'],
451                     game_score=row['game_score'],
452                     server=row['server'],
453                     ending_type=row['ending_type']
454                 ))
455         return points
456
457     def get_cricket_deliveries(self, video_id: str) -> List[CricketDelivery]:
458         deliveries = []
459         with self._get_connection() as conn:
460             cursor = conn.cursor()
461             cursor.execute("""
462                 SELECT * FROM cricket_deliveries
463                 WHERE video_id = ?
464                 ORDER BY over_num, ball_num

```



```

465         """", (video_id,))
466     for row in cursor.fetchall():
467         deliveries.append(CricketDelivery(
468             video_id=row['video_id'],
469             over=row['over_num'],
470             ball=row['ball_num'],
471             start_time=row['start_time'],
472             end_time=row['end_time'],
473             batsman=row['batsman'],
474             bowler=row['bowler'],
475             runs=row['runs'],
476             wicket=bool(row['wicket']),
477             extras=row['extras']
478         ))
479     return deliveries
480
481 def get_pickleball_rallies(self, video_id: str) -> List[PickleballRally]:
482     rallies = []
483     with self._get_connection() as conn:
484         cursor = conn.cursor()
485         cursor.execute("""
486         SELECT * FROM pickleball_rallies
487         WHERE video_id = ?
488         ORDER BY rally_number
489         """, (video_id,))
490     for row in cursor.fetchall():
491         rallies.append(PickleballRally(
492             video_id=row['video_id'],
493             rally_number=row['rally_number'],
494             start_time=row['start_time'],
495             end_time=row['end_time'],
496             score_before=row['score_before'],
497             score_after=row['score_after'],
498             shots_count=row['shots_count'],
499             ending_reason=row['ending_reason'],
500             last_shot_outcome=row['last_shot_outcome']
501         ))
502     return rallies
503
504 def get_tabletennis_rallies(self, video_id: str) -> List[TableTennisRally]:
505     rallies = []
506     with self._get_connection() as conn:
507         cursor = conn.cursor()
508         cursor.execute("""
509         SELECT * FROM tabletennis_rallies
510         WHERE video_id = ?
511         ORDER BY rally_number
512         """, (video_id,))
513     for row in cursor.fetchall():
514         rallies.append(TableTennisRally(
515             video_id=row['video_id'],
516             rally_number=row['rally_number'],
517             start_time=row['start_time'],
518             end_time=row['end_time'],
519             score_before=row['score_before'],
520             score_after=row['score_after'],
521             shots_count=row['shots_count'],
522             ending_reason=row['ending_reason'],
523             last_shot_outcome=row['last_shot_outcome']
524         ))
525     return rallies
526
527 def get_segment_intervals(self, video_id: str, sport: SportType) -> List[tuple]:
528     """Return chronologically-ordered (start_time, end_time) tuples for a video,
529     regardless of sport. This is the sport-agnostic view used to export

```

```

530         indexer-ready evaluation/training labels.
531
532     Sports that have no annotation table yet (e.g. tabletennis, pickleball)
533     yield an empty list rather than raising.
534     """
535     mapping = SEGMENT_TABLES.get(sport)
536     if mapping is None:
537         return []
538     table, order_by = mapping # both come from the trusted map above, never user
input
539     with self._get_connection() as conn:
540         cursor = conn.cursor()
541         cursor.execute(
542             f"SELECT start_time, end_time FROM {table} "
543             f"WHERE video_id = ? ORDER BY {order_by}",
544             (video_id,),
545         )
546         return [(row["start_time"], row["end_time"]) for row in cursor.fetchall()]
547
548     def update_status(self, video_id: str, status: CurationStatus) -> bool:
549         with self._get_connection() as conn:
550             cursor = conn.cursor()
551             cursor.execute("UPDATE videos SET status = ? WHERE video_id = ?",
(status.value, video_id))
552             conn.commit()
553             return cursor.rowcount > 0
554
555     def delete_video(self, video_id: str) -> bool:
556         """Deletes a video and cascading child entries."""
557         with self._get_connection() as conn:
558             cursor = conn.cursor()
559             # Execute cascade deletes manually just in case SQLite foreign key support
is not active
560             cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
561             cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
562             cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))
563             cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
(video_id,))
564             cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))
565             cursor.execute("DELETE FROM videos WHERE video_id = ?", (video_id,))
566             conn.commit()
567             return cursor.rowcount > 0
568

```

4. user (2026-06-08T08:39:04.261Z)

```

1     """Adapt collector data into an obsreport envelope.
2
3     Reuses the existing rally_cvat report primitives (`Issue`, records, severities)
4     rather than re-deriving anything ? this module only *maps* them onto the shared
5     report shape. Imported only after `src.reporting` confirms obsreport is present,
6     so it may import obsreport freely.
7     """
8     from __future__ import annotations
9
10    from pathlib import Path
11    from typing import Any, List, Optional
12
13    import obsreport
14
15    _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
16

```

```

17 # Delivery issue severity -> report flag severity.
18 _SEVERITY = {"blocker": "error", "high": "warn", "medium": "warn", "low": "info"}
19 # Every issue code is documented in this FAQ table.
20 _ISSUE_FAQ = "docs/INGESTION_PIPELINE.md#what-does-validation-check"
21
22
23 def load_spec() -> "obsreport.ReportSpec":
24     return obsreport.load_spec(_SPEC_PATH)
25
26
27 # Heuristic checks that flag a region to REVIEW ? not hard failures. We frame these
28 # gently for the vendor so a "possible" finding isn't read as a certainty/blocker.
29 _REVIEW_ONLY = {"possible_missed_rally", "shot_density"}
30
31
32 def issue_to_flag(issue: Any, record: Any = None, fps: Any = None) -> "obsreport.Flag":
33     """Map a rally_cvat.Issue onto a report Flag. Actionable issues (blocker/high/
34     medium) also carry a customer_message so the delivery summary shows the vendor
35     exactly what to fix, in plain language. When the rally's source record + fps are
36     known, frame-level evidence is attached so a CVAT (frame-based) reviewer can act
37     without converting seconds to frames by hand."""
38     sev = _SEVERITY.get(issue.severity, "info")
39     customer = issue.message if sev in ("error", "warn") else None
40     if customer and issue.code in _REVIEW_ONLY:
41         customer += (" (Review only ? a heuristic, not a blocker: check the video and "
42                     "act only if confirmed.)")
43     evidence: dict = {}
44     if issue.rally_number is not None:
45         evidence["rally_number"] = issue.rally_number
46     if record is not None and getattr(record, "frame_min", -1) is not None and
47     getattr(record, "frame_min", -1) >= 0:
48         evidence["rally_frames"] = f"{record.frame_min}-{record.frame_max}"
49     if fps:
50         evidence["fps"] = fps
51     return obsreport.Flag(
52         code=issue.code,
53         severity=sev,
54         message=issue.message,
55         stage="validation",
56         faq_ref=_ISSUE_FAQ,
57         customer_message=customer,
58         evidence=evidence,
59     )
60
61 def _recorder(source: str, video_path: Optional[str], spec) -> "obsreport.RunRecorder":
62     return obsreport.RunRecorder(
63         "sports-data-collector",
64         video_id=source,
65         video_id_kind="human_id",
66         video_path=video_path,
67         spec=spec if spec is not None else load_spec(),
68     )
69
70
71 def build_delivery_recorder(
72     *,
73     records: List[Any],
74     issues: List[Any],
75     fps: Optional[float],
76     source: str,
77     video_path: Optional[str] = None,
78     video_duration_s: Optional[float] = None,
79     source_format: str = "cvat",
80     step: Optional[int] = None,

```

```

81         spec=None,
82     ) -> "obsreport.RunRecorder":
83         """Envelope for `validate-delivery` / `convert-cvat`: parse + validation stages,
84         one flag per delivery issue, a rally summary."""
85         rec = _recorder(source, video_path, spec)
86         if fps:
87             rec.set_video_meta(fps=fps, fps_source="declared")
88         if video_duration_s:
89             rec.set_video_meta(duration_s=round(video_duration_s, 1))
90
91         with rec.stage("parse") as st:
92             st.add_metric("rallies_parsed", len(records))
93             st.details.update({"source_format": source_format, "fps": fps, "sampling_step":
step})
94
95         blockers = [i for i in issues if i.severity == "blocker"]
96         with rec.stage("validation") as st:
97             for sev in ("blocker", "high", "medium", "low"):
98                 st.add_metric(f"issues_{sev}", sum(1 for i in issues if i.severity == sev))
99             st.add_metric("n_blockers", len(blockers))
100            st.details["gate"] = "BLOCKED" if blockers else "PASS"
101            st.status = "error" if blockers else ("warn" if issues else "ok")
102
103            records_by_rally = {getattr(r, "rally_number", None): r for r in records}
104            for issue in issues:
105                record = records_by_rally.get(issue.rally_number)
106                rec.envelope.flags.append(issue_to_flag(issue, record=record, fps=fps))
107
108            _rally_summary(rec, records, video_duration_s)
109            rec.finalize()
110            return rec
111
112
113     def build_import_recorder(
114         *,
115         video_meta_obj: Any,
116         annotations: List[Any],
117         is_valid: bool,
118         errors: List[str],
119         count: int,
120         sport: str,
121         source_format: str,
122         spec=None,
123     ) -> "obsreport.RunRecorder":
124         """Envelope for `import-local`: parse + validation + registration stages."""
125         rec = _recorder(video_meta_obj.video_id, video_meta_obj.local_path, spec)
126         fps = getattr(video_meta_obj, "fps", None)
127         rec.set_video_meta(
128             fps=fps,
129             fps_source="declared" if fps else "unknown",
130             resolution=getattr(video_meta_obj, "resolution", None),
131         )
132
133         with rec.stage("parse") as st:
134             st.add_metric("records_parsed", len(annotations))
135             st.details["source_format"] = source_format
136
137         with rec.stage("validation") as st:
138             st.status = "ok" if is_valid else "error"
139             st.add_metric("valid", is_valid)
140             for e in errors or []:
141                 st.messages.append(str(e))
142
143         with rec.stage("registration") as st:
144             st.add_metric("segments_registered", count)

```

```

145         st.details.update({
146             "sport": sport,
147             "license": getattr(getattr(video_meta_obj, "license_type", None), "value",
148                               str(getattr(video_meta_obj, "license_type", ""))),
149             "is_commercial": bool(getattr(video_meta_obj, "is_commercial", False)),
150             "status": getattr(getattr(video_meta_obj, "status", None), "value",
151                               str(getattr(video_meta_obj, "status", ""))),
152         })
153
154     _rally_summary(rec, annotations, None)
155     rec.finalize()
156     return rec
157
158
159     def _rally_summary(rec: "obsreport.RunRecorder", records: List[Any], video_duration_s:
Optional[float]) -> None:
160         total = 0.0
161         for r in records:
162             try:
163                 total += float(r.end_time) - float(r.start_time)
164             except (AttributeError, TypeError, ValueError):
165                 pass
166         coverage = round(100.0 * total / video_duration_s, 1) if video_duration_s else None
167         notes = [] if records else ["No rallies were found in this delivery."]
168         rec.set_highlights(segment_count=len(records), total_highlight_s=round(total, 1),
169                           coverage_pct=coverage, notes=notes)
170

```

5. user (2026-06-08T08:39:04.492Z)

```

1     # Observability report spec for the sports-data-collector.
2     # The report ENGINE lives in the shared `obsreport` library; this is the
3     # collector-specific layout + a couple of rules. Most delivery findings come
4     # straight from rally_cvat's Issue list (mapped 1:1 to flags by the adapter);
5     # the rules below cover signals that aren't delivery "issues".
6     name: collector
7     title: Sports Data Collector ? Delivery / Ingestion Report
8
9     # Customer here = the annotation vendor / data team receiving delivery feedback.
10    customer_verdict:
11        pass: "This delivery passed all checks and is ready to import."
12        pass_with_warnings: "This delivery is importable, but please review the flagged
rallies above."
13        fail: "This delivery has blocking issues that must be fixed before it can be imported
? see 'Things to review'."
14
15    sections:
16        - { id: run,          title: "Run",          kind: identity,  audience: both }
17        - { id: video,       title: "Video",         kind: video_meta, audience: both }
18        - { id: summary,    title: "Rally summary",   kind: highlights, audience: customer,
item_noun: rally }
19        - { id: review,     title: "Things to review", kind: flags,      audience: customer
}
20        - { id: stages,     title: "Processing stages", kind: stages,     audience: technical
}
21        - { id: findings,   title: "Findings & issues", kind: flags,      audience: technical
}
22        - { id: verdict,    title: "Verdict",         kind: verdict,    audience: both }
23
24    rules:
25        # A non-commercial license means the video is kept but EXCLUDED from the default
26        # (commercial-only) export ? easy to forget when it later "goes missing" downstream.
27        - code: license_noncommercial
28          severity: warn
29          when:

```

```

30         - { path: "stages.registration.details.is_commercial", op: eq, value: false }
31     message: >-
32         This video's license is non-commercial, so it is EXCLUDED from the default
33         commercial-only export. It stays in the DB; pass --include-noncommercial to
34         export it.
35     faq_ref: "docs/INGESTION_PIPELINE.md#will-a-non-commercial-license-block-export"
36
37     # No fps recorded at import -> downstream export/manifest carries no fps and the
38     # indexer must probe the file; provide --fps for accurate frame<->second timing.
39     - code: missing_fps
40     severity: warn
41     when:
42         - { path: "video_meta.fps", op: eq, value: null }
43         - { path: "stages.registration.metrics.segments_registered", op: is_present }
44     message: >-
45         No fps was recorded for this video. The export manifest will lack fps and the
46         indexer will have to probe the file. Pass --fps for accurate timing.
47     faq_ref: "docs/INGESTION_PIPELINE.md#how-are-timestamps-handled-the-fps-thing"
48

```

6. user (2026-06-08T08:39:04.758Z)

```

1     """Per-video observability reports for the collector.
2
3     `obsreport` is a SOFT dependency: absent -> these emitters no-op and the CLI runs
4     unchanged. Every emitter is safe to call from normal control flow and NEVER raises
5     (emitting a report must not break ingestion). Returns the written paths, or None.
6     """
7     from __future__ import annotations
8
9     import os
10    from typing import Any, Dict, List, Optional
11
12    try: # soft dependency
13        import obsreport # noqa: F401
14        from . import adapter
15
16        OBSREPORT_AVAILABLE = True
17    except Exception: # pragma: no cover - only when obsreport is absent
18        OBSREPORT_AVAILABLE = False
19
20
21    def emit_delivery_report(
22        *,
23        out_dir: str,
24        source: str,
25        records: List[Any],
26        issues: List[Any],
27        fps: Optional[float],
28        video_path: Optional[str] = None,
29        video_duration_s: Optional[float] = None,
30        source_format: str = "cvat",
31        step: Optional[int] = None,
32    ) -> Optional[Dict[str, str]]:
33        """Write the delivery observability report into out_dir (alongside the existing
34        <id>_report.* artifacts). Used by validate-delivery / convert-cvat. Never raises."""
35        if not OBSREPORT_AVAILABLE:
36            return None
37        try:
38            rec = adapter.build_delivery_recorder(
39                records=records, issues=issues, fps=fps, source=source,
40                video_path=video_path,
41                video_duration_s=video_duration_s, source_format=source_format, step=step,
42            )
43            res = rec.write(output_dir=out_dir, stem=source)

```

```

43         return res.paths or None
44     except Exception: # pragma: no cover - reporting must never break the command
45         return None
46
47
48     def emit_import_report(
49         *,
50         video_meta_obj: Any,
51         annotations: List[Any],
52         is_valid: bool,
53         errors: List[str],
54         count: int,
55         sport: str,
56         source_format: str,
57         fallback_dir: Optional[str] = None,
58     ) -> Optional[Dict[str, str]]:
59         """Write the import observability report next to the imported video. If the video
60         file doesn't exist on disk (registration-only imports), fall back to
61         ``fallback_dir`` (e.g. the annotations directory) rather than the cwd. Never
62         raises."""
63         if not OBSREPORT_AVAILABLE:
64             return None
65         try:
66             rec = adapter.build_import_recorder(
67                 video_meta_obj=video_meta_obj, annotations=annotations, is_valid=is_valid,
68                 errors=errors, count=count, sport=sport, source_format=source_format,
69             )
70             video_path = getattr(video_meta_obj, "local_path", None)
71             if video_path and os.path.exists(video_path):
72                 res = rec.write() # next to the video
73             else:
74                 res = rec.write(output_dir=fallback_dir or ".",
75                                stem=getattr(video_meta_obj, "video_id", None))
76             return res.paths or None
77         except Exception: # pragma: no cover
78             return None

```

7. user (2026-06-08T08:39:04.999Z)

```

1     """Integration tests for the import-local annotation parser on the newly-added
2     sports (pickleball, table tennis). _parse_local_annotations does not touch `self`,
3     so we can exercise it directly with a temp file and no DB side effects."""
4     import json
5     import csv as _csv
6
7     from src.cli.curate import SportsDataCLI
8     from src.core.schemas.pickleball import PickleballRally
9     from src.core.schemas.tabletennis import TableTennisRally
10
11
12     def _parse(sport, video_id, path):
13         # self is unused inside _parse_local_annotations, so None is safe here.
14         return SportsDataCLI._parse_local_annotations(None, sport, video_id, str(path))
15
16
17     def test_parse_pickleball_csv(tmp_path):
18         p = tmp_path / "pb.csv"
19         with open(p, "w", newline="", encoding="utf-8") as f:
20             w = _csv.writer(f)
21             w.writerow(["rally_number", "start_time", "end_time", "shots_count",
22 "ending_reason", "score_before", "score_after", "notes"])
23             w.writerow([1, 12.50, 25.00, 6, "winner", "0-0", "1-0", ""])
24             w.writerow([2, 30.00, 31.20, 1, "service_fault", "1-0", "1-1", "serve into
25 net"])

```

```

24
25     items = _parse("pickleball", "pb_csv", p)
26     assert len(items) == 2
27     assert all(isinstance(i, PickleballRally) for i in items)
28     assert items[0].shots_count == 6
29     assert items[1].ending_reason == "service_fault"
30     # Unknown columns (notes) are ignored, not fatal.
31     assert items[0].start_time == 12.5
32
33
34     def test_parse_tabletennis_json(tmp_path):
35         p = tmp_path / "tt.json"
36         data = [
37             {"rally_number": 1, "start_time": 5.0, "end_time": 9.0, "shots_count": 7,
"ending_reason": "winner"},
38             {"rally_number": 2, "start_time": 12.0, "end_time": 13.0, "shots_count": 1,
"ending_reason": "let"},
39         ]
40         p.write_text(json.dumps(data), encoding="utf-8")
41
42         items = _parse("tabletennis", "tt_json", p)
43         assert len(items) == 2
44         assert all(isinstance(i, TableTennisRally) for i in items)
45         assert items[1].ending_reason == "let"
46         assert items[0].video_id == "tt_json"
47

```

8. user (2026-06-08T08:39:05.014Z)

```

1     import pytest
2     from src.core.validator import DatasetValidator
3     from src.core.schemas.base import LicenseType
4     from src.core.schemas.badminton import BadmintonRally
5     from src.core.schemas.tennis import TennisPoint
6     from src.core.schemas.cricket import CricketDelivery
7     from src.core.schemas.pickleball import PickleballRally
8     from src.core.schemas.tabletennis import TableTennisRally
9
10    @pytest.fixture
11    def validator():
12        # Use default configs/fallback
13        return DatasetValidator(config_path="nonexistent_config.yaml")
14
15    def test_license_checks(validator):
16        # Fallback list has MIT, Apache-2.0, BSD, CC0, Public-Domain, Proprietary
17        assert validator.check_license_commercial(LicenseType.MIT) is True
18        assert validator.check_license_commercial(LicenseType.APACHE_2) is True
19        assert validator.check_license_commercial(LicenseType.PROPRIETARY) is True
20        assert validator.check_license_commercial(LicenseType.CC_BY_NC) is False
21        assert validator.check_license_commercial(LicenseType.CC_BY_NC_SA) is False
22
23    def test_validate_badminton_rallies(validator):
24        # Valid sequential rallies
25        valid_rallies = [
26            BadmintonRally(video_id="v1", rally_number=1, start_time=10.0, end_time=20.0),
27            BadmintonRally(video_id="v1", rally_number=2, start_time=25.0, end_time=35.0),
28        ]
29        ok, errors = validator.validate_badminton_rallies(valid_rallies)
30        assert ok is True
31        assert len(errors) == 0
32
33        # Overlapping rallies
34        invalid_rallies = [
35            BadmintonRally(video_id="v1", rally_number=1, start_time=10.0, end_time=20.0),
36            BadmintonRally(video_id="v1", rally_number=2, start_time=18.0, end_time=30.0),

```



```

37     ]
38     ok, errors = validator.validate_badminton_rallies(invalid_rallies)
39     assert ok is False
40     assert len(errors) > 0
41     assert "overlap" in errors[0]
42
43 def test_validate_tennis_points(validator):
44     valid = [
45         TennisPoint(video_id="v3", point_number=1, start_time=10.0, end_time=20.0),
46         TennisPoint(video_id="v3", point_number=2, start_time=25.0, end_time=35.0),
47     ]
48     ok, errors = validator.validate_tennis_points(valid)
49     assert ok is True
50     assert len(errors) == 0
51
52     invalid = [
53         TennisPoint(video_id="v3", point_number=1, start_time=10.0, end_time=20.0),
54         TennisPoint(video_id="v3", point_number=2, start_time=18.0, end_time=30.0),
55     ]
56     ok, errors = validator.validate_tennis_points(invalid)
57     assert ok is False
58     assert "overlap" in errors[0]
59
60
61 def test_validate_empty_lists_are_ok(validator):
62     assert validator.validate_badminton_rallies([]) == (True, [])
63     assert validator.validate_pickleball_rallies([]) == (True, [])
64     assert validator.validate_tabletennis_rallies([]) == (True, [])
65     assert validator.validate_tennis_points([]) == (True, [])
66     assert validator.validate_cricket_deliveries([]) == (True, [])
67
68
69 def test_validate_cricket_deliveries(validator):
70     valid_delivs = [
71         CricketDelivery(video_id="v2", over=0, ball=1, start_time=5.0, end_time=15.0),
72         CricketDelivery(video_id="v2", over=0, ball=2, start_time=20.0, end_time=30.0)
73     ]
74     ok, errors = validator.validate_cricket_deliveries(valid_delivs)
75     assert ok is True
76
77     invalid_delivs = [
78         CricketDelivery(video_id="v2", over=0, ball=1, start_time=5.0, end_time=15.0),
79         CricketDelivery(video_id="v2", over=0, ball=2, start_time=12.0, end_time=22.0)
80     ]
81     ok, errors = validator.validate_cricket_deliveries(invalid_delivs)
82     assert ok is False
83     assert len(errors) == 1
84
85 def test_validate_pickleball_rallies(validator):
86     valid = [
87         PickleballRally(video_id="p1", rally_number=1, start_time=10.0, end_time=20.0),
88         PickleballRally(video_id="p1", rally_number=2, start_time=25.0, end_time=35.0),
89     ]
90     ok, errors = validator.validate_pickleball_rallies(valid)
91     assert ok is True
92     assert len(errors) == 0
93
94     invalid = [
95         PickleballRally(video_id="p1", rally_number=1, start_time=10.0, end_time=20.0),
96         PickleballRally(video_id="p1", rally_number=2, start_time=18.0, end_time=30.0),
97     ]
98     ok, errors = validator.validate_pickleball_rallies(invalid)
99     assert ok is False
100    assert "overlap" in errors[0]
101

```

```

102     def test_validate_tabletennis_rallies validator:
103         valid = [
104             TableTennisRally(video_id="t1", rally_number=1, start_time=5.0, end_time=9.0),
105             TableTennisRally(video_id="t1", rally_number=2, start_time=12.0, end_time=13.0),
106         ]
107         ok, errors = validator.validate_tabletennis_rallies(valid)
108         assert ok is True
109         assert len(errors) == 0
110
111         invalid = [
112             TableTennisRally(video_id="t1", rally_number=1, start_time=5.0, end_time=9.0),
113             TableTennisRally(video_id="t1", rally_number=2, start_time=8.0, end_time=13.0),
114         ]
115         ok, errors = validator.validate_tabletennis_rallies(invalid)
116         assert ok is False
117         assert "overlap" in errors[0]
118

```

9. assistant (2026-06-08T08:39:07.498Z)

Now let me search for more context on how imports are handled and what schemas define the maturity/status fields.

10. user (2026-06-08T08:39:08.023Z)

```

C:/Users/avidu/Projects/sports-data-collector/src/core/schemas\tennis.py
C:/Users/avidu/Projects/sports-data-collector/src/core/schemas\cricket.py
C:/Users/avidu/Projects/sports-data-collector/src/core/schemas\base.py
C:/Users/avidu/Projects/sports-data-collector/src/core/schemas\badminton.py
C:/Users/avidu/Projects/sports-data-collector/src/core/schemas\pickleball.py
C:/Users/avidu/Projects/sports-data-collector/src/core/schemas\tabletennis.py

```

11. user (2026-06-08T08:39:08.302Z)

```

C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py
C:/Users/avidu/Projects/sports-data-collector/src/cli/ingest.py

```

12. assistant (2026-06-08T08:39:11.201Z)

Let me read the base schemas and the CLI import handler:

13. user (2026-06-08T08:39:11.456Z)

```

1     from pydantic import BaseModel, Field
2     from typing import Optional
3     from enum import Enum
4
5     class SportType(str, Enum):
6         BADMINTON = "badminton"
7         TENNIS = "tennis"
8         TABLE_TENNIS = "tabletennis"
9         PICKLEBALL = "pickleball"
10        CRICKET = "cricket"
11
12    class LicenseType(str, Enum):
13        MIT = "MIT"
14        APACHE_2 = "Apache-2.0"
15        BSD = "BSD"
16        CC0 = "CC0"
17        CC_BY = "CC-BY"
18        CC_BY_NC = "CC-BY-NC"
19        CC_BY_NC_SA = "CC-BY-NC-SA"
20        PUBLIC_DOMAIN = "Public-Domain"
21        PROPRIETARY = "Proprietary"

```

```

22         UNKNOWN = "Unknown"
23
24     class CurationStatus(str, Enum):
25         STAGING = "staging"
26         CURATED = "curated"
27         REJECTED = "rejected"
28
29     class VideoMetadata(BaseModel):
30         video_id: str = Field(..., description="Unique alphanumeric identifier for the
video")
31         sport: SportType
32         video_url: Optional[str] = Field(None, description="Remote URL of the video (if
available)")
33         local_path: Optional[str] = Field(None, description="Local path to video file (if
downloaded or custom)")
34         license_type: LicenseType
35         license_url: Optional[str] = None
36         is_commercial: bool = Field(False, description="Flag indicating if the license
allows commercial use")
37         status: CurationStatus = CurationStatus.STAGING
38         match_name: str = Field(..., description="Name of the match/event")
39         fps: Optional[float] = Field(None, description="Frames per second")
40         resolution: Optional[str] = Field(None, description="Resolution string (e.g. 1080p,
720p)")
41

```

14. user (2026-06-08T08:39:11.757Z)

```

1     import argparse
2     import sys
3     from src.core.config import load_config, get_database_path
4     from src.core.db import SportsDatabase
5     from src.parsers.badminton.coachai import CoachAIBadmintonParser
6
7     def main():
8         parser = argparse.ArgumentParser(description="Ingest external datasets into the
curation staging area")
9         parser.add_argument("--sport", required=True, choices=["badminton"], help="Sport
type")
10        parser.add_argument("--parser", required=True, choices=["coachai"], help="Parser
type to use")
11        parser.add_argument("--file-path", required=True, help="Path to raw dataset file")
12        parser.add_argument("--license", default="MIT", help="Dataset license type")
13        parser.add_argument("--video-url", default="https://youtube.com/watch?v=example",
help="URL to the video")
14        parser.add_argument("--match-name", help="Name of the match")
15        parser.add_argument("--video-id", help="Video unique identifier")
16
17        args = parser.parse_args()
18
19        # Load db path from the central config (falls back to the default).
20        # NOTE: this importer is intentionally badminton/CoachAI-only for now; other
21        # sports are populated via `curate import-local` or vendor golden sets.
22        db = SportsDatabase(get_database_path(load_config()))
23
24        if args.sport == "badminton" and args.parser == "coachai":
25            match_parser = CoachAIBadmintonParser()
26            try:
27                video_meta, rallies = match_parser.parse(
28                    args.file_path,
29                    video_id=args.video_id,
30                    video_url=args.video_url,
31                    license=args.license,
32                    match_name=args.match_name
33                )

```

```

34
35         # Insert into database (by default parsed entries are marked as staging)
36         db.insert_video(video_meta)
37         count = db.insert_badminton_rallies(rallies)
38
39         print(f"Successfully ingested match '{video_meta.match_name}' into
staging:")
40         print(f" - Video ID: {video_meta.video_id}")
41         print(f" - Total Rallies: {count}")
42         print(f" - License: {video_meta.license_type.value}")
43         print(f" - Commercial safe: {video_meta.is_commercial}")
44     except Exception as e:
45         print(f"Error parsing file: {e}")
46         sys.exit(1)
47
48     if __name__ == "__main__":
49         main()
50

```

15. user (2026-06-08T08:39:12.181Z)

```

1     import os
2     import sys
3     import time
4     import json
5     import csv
6     import argparse
7     from typing import List, Dict, Any, Tuple
8     from tabulate import tabulate
9
10    from src.core.config import load_config
11    from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12    from src.core.db import SportsDatabase
13    from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14    from src.core.schemas.badminton import BadmintonRally
15    from src.core.schemas.tennis import TennisPoint
16    from src.core.schemas.cricket import CricketDelivery
17    from src.core.schemas.pickleball import PickleballRally
18    from src.core.schemas.tabletennis import TableTennisRally
19    from src.core.validator import DatasetValidator
20    from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23    class SportsDataCLI:
24        def __init__(self, config_path: str = "config/config.yaml"):
25            self.config_path = config_path
26            self.config = self._load_config()
27
28            # Resolve DB path
29            db_path = self.config.get("database_path", "data/sports_metadata.db")
30            self.db = SportsDatabase(db_path)
31            self.validator = DatasetValidator(config_path)
32
33        def _load_config(self) -> Dict[str, Any]:
34            if not os.path.exists(self.config_path):
35                print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36            return load_config(self.config_path)
37
38        def status(self):
39            """Displays status summary of all videos in the database."""
40            # Query total count by status via the public DB read API.
41            rows = self.db.get_status_counts() # list of (sport, status, cnt)
42

```

```

43         if not rows:
44             print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45             return
46
47         data = [[sport, status, cnt] for sport, status, cnt in rows]
48         print("\n=== Dataset Ingestion Status ===")
49         print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50         print()
51
52     def import_local(self, args):
53         """Manually registers a local video and annotation file into the database."""
54         sport_str = args.sport.lower()
55         if sport_str not in self.config.get("supported_sports", ["badminton"]):
56             print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57             sys.exit(1)
58
59         # Validate video path
60         if not os.path.exists(args.video_path):
61             print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63         # Validate annotation file
64         if not os.path.exists(args.annotations_path):
65             print(f"Error: Annotations file '{args.annotations_path}' not found.")
66             sys.exit(1)
67
68         match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69         video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71         # Parse license
72         try:
73             license_type = LicenseType(args.license)
74         except ValueError:
75             print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76             license_type = LicenseType.UNKNOWN
77
78         is_commercial = self.validator.check_license_commercial(license_type)
79
80         # 1. Parse Annotations
81         annotations = []
82         try:
83             annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84         except Exception as e:
85             print(f"Error parsing annotations: {e}")
86             sys.exit(1)
87
88         # 2. Run Validation
89         is_valid = True
90         errors = []
91         if sport_str == "badminton":
92             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93         elif sport_str == "tennis":
94             is_valid, errors = self.validator.validate_tennis_points(annotations)
95         elif sport_str == "cricket":
96             is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97         elif sport_str == "pickleball":
98             is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99         elif sport_str == "tabletennis":
100             is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102         if not is_valid:
103             print(f"Error: Annotations failed validation!")

```

```

104         for err in errors:
105             print(f" - {err}")
106         sys.exit(1)
107
108     # 3. Create Video Metadata
109     video_meta = VideoMetadata(
110         video_id=video_id,
111         sport=SportType(sport_str),
112         video_url=None,
113         local_path=args.video_path,
114         license_type=license_type,
115         license_url=None,
116         is_commercial=is_commercial,
117         status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
118         match_name=match_name,
119         fps=args.fps,
120         resolution=args.resolution
121     )
122
123     # 4. Insert into Database
124     self.db.insert_video(video_meta)
125
126     count = 0
127     if sport_str == "badminton":
128         count = self.db.insert_badminton_rallies(annotations)
129     elif sport_str == "tennis":
130         count = self.db.insert_tennis_points(annotations)
131     elif sport_str == "cricket":
132         count = self.db.insert_cricket_deliveries(annotations)
133     elif sport_str == "pickleball":
134         count = self.db.insert_pickleball_rallies(annotations)
135     elif sport_str == "tabletennis":
136         count = self.db.insert_tabletennis_rallies(annotations)
137
138     print(f"\nSuccessfully imported local video '{match_name}':")
139     print(f" - Video ID: {video_id}")
140     print(f" - Sport: {sport_str}")
141     print(f" - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142     print(f" - Registered {count} annotated segments.\n")
143
144     # Per-video observability report (next to the video). Never raises.
145     from src.reporting import emit_import_report
146     src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
147     report_paths = emit_import_report(
148         video_meta_obj=video_meta, annotations=annotations, is_valid=True,
149         errors=[], count=count, sport=sport_str, source_format=src_fmt,
150         fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
151     )
152     if report_paths:
153         print(f" - Observability report: {report_paths.get('technical_md')}")
154         print(f" - Customer summary      : {report_paths.get('customer_md')}\n")
155
156     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
157         """Parses simple JSON or CSV annotations files."""
158         parsed_items = []
159         ext = os.path.splitext(filepath)[1].lower()
160
161         if ext == '.json':
162             with open(filepath, 'r', encoding='utf-8') as f:
163                 data = json.load(f)
164

```

```

165         if not isinstance(data, list):
166             raise ValueError("JSON file must contain a list of objects.")
167
168         # Uses the same safe casts as the CSV path: a missing/blank required
169         # time raises a clear ValueError (caught by the caller) instead of a
170         # bare KeyError, and optional ints degrade to sensible defaults.
171         for idx, obj in enumerate(data):
172             if sport == "badminton":
173                 parsed_items.append(BadmintonRally(
174                     video_id=video_id,
175                     rally_number=_safe_int(obj.get("rally_number"), idx + 1),
176                     start_time=_safe_float(obj.get("start_time")),
177                     end_time=_safe_float(obj.get("end_time")),
178                     score_before=obj.get("score_before"),
179                     score_after=obj.get("score_after"),
180                     shots_count=_safe_int(obj.get("shots_count"), None),
181                     ending_reason=obj.get("ending_reason"),
182                     last_shot_outcome=obj.get("last_shot_outcome")
183                 ))
184             elif sport == "tennis":
185                 parsed_items.append(TennisPoint(
186                     video_id=video_id,
187                     point_number=_safe_int(obj.get("point_number"), idx + 1),
188                     start_time=_safe_float(obj.get("start_time")),
189                     end_time=_safe_float(obj.get("end_time")),
190                     set_score=obj.get("set_score"),
191                     game_score=obj.get("game_score"),
192                     server=obj.get("server"),
193                     ending_type=obj.get("ending_type")
194                 ))
195             elif sport == "cricket":
196                 parsed_items.append(CricketDelivery(
197                     video_id=video_id,
198                     over=_safe_int(obj.get("over"), 0),
199                     ball=_safe_int(obj.get("ball"), 0),
200                     start_time=_safe_float(obj.get("start_time")),
201                     end_time=_safe_float(obj.get("end_time")),
202                     batsman=obj.get("batsman"),
203                     bowler=obj.get("bowler"),
204                     runs=_safe_int(obj.get("runs"), None),
205                     wicket=bool(obj.get("wicket", False)),
206                     extras=_safe_int(obj.get("extras"), 0)
207                 ))
208             elif sport == "pickleball":
209                 parsed_items.append(PickleballRally(
210                     video_id=video_id,
211                     rally_number=_safe_int(obj.get("rally_number"), idx + 1),
212                     start_time=_safe_float(obj.get("start_time")),
213                     end_time=_safe_float(obj.get("end_time")),
214                     score_before=obj.get("score_before"),
215                     score_after=obj.get("score_after"),
216                     shots_count=_safe_int(obj.get("shots_count"), None),
217                     ending_reason=obj.get("ending_reason"),
218                     last_shot_outcome=obj.get("last_shot_outcome")
219                 ))
220             elif sport == "tabletennis":
221                 parsed_items.append(TableTennisRally(
222                     video_id=video_id,
223                     rally_number=_safe_int(obj.get("rally_number"), idx + 1),
224                     start_time=_safe_float(obj.get("start_time")),
225                     end_time=_safe_float(obj.get("end_time")),
226                     score_before=obj.get("score_before"),
227                     score_after=obj.get("score_after"),
228                     shots_count=_safe_int(obj.get("shots_count"), None),
229                     ending_reason=obj.get("ending_reason"),

```

```

230         last_shot_outcome=obj.get("last_shot_outcome")
231     ))
232     elif ext == '.csv':
233         with open(filepath, 'r', encoding='utf-8') as f:
234             reader = csv.DictReader(f)
235             # Normalize headers
236             reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
237
238         for idx, row in enumerate(reader):
239             if sport == "badminton":
240                 parsed_items.append(BadmintonRally(
241                     video_id=video_id,
242                     rally_number=_safe_int(row.get("rally_number"), idx + 1),
243                     start_time=_safe_float(row.get("start_time")),
244                     end_time=_safe_float(row.get("end_time")),
245                     score_before=row.get("score_before"),
246                     score_after=row.get("score_after"),
247                     shots_count=_safe_int(row.get("shots_count"), None),
248                     ending_reason=row.get("ending_reason"),
249                     last_shot_outcome=row.get("last_shot_outcome")
250                 ))
251             elif sport == "tennis":
252                 parsed_items.append(TennisPoint(
253                     video_id=video_id,
254                     point_number=_safe_int(row.get("point_number"), idx + 1),
255                     start_time=_safe_float(row.get("start_time")),
256                     end_time=_safe_float(row.get("end_time")),
257                     set_score=row.get("set_score"),
258                     game_score=row.get("game_score"),
259                     server=row.get("server"),
260                     ending_type=row.get("ending_type")
261                 ))
262             elif sport == "cricket":
263                 parsed_items.append(CricketDelivery(
264                     video_id=video_id,
265                     over=_safe_int(row.get("over"), 0),
266                     ball=_safe_int(row.get("ball"), 0),
267                     start_time=_safe_float(row.get("start_time")),
268                     end_time=_safe_float(row.get("end_time")),
269                     batsman=row.get("batsman"),
270                     bowler=row.get("bowler"),
271                     runs=_safe_int(row.get("runs"), None),
272                     wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
273                     extras=_safe_int(row.get("extras"), 0)
274                 ))
275             elif sport == "pickleball":
276                 parsed_items.append(PickleballRally(
277                     video_id=video_id,
278                     rally_number=_safe_int(row.get("rally_number"), idx + 1),
279                     start_time=_safe_float(row.get("start_time")),
280                     end_time=_safe_float(row.get("end_time")),
281                     score_before=row.get("score_before"),
282                     score_after=row.get("score_after"),
283                     shots_count=_safe_int(row.get("shots_count"), None),
284                     ending_reason=row.get("ending_reason"),
285                     last_shot_outcome=row.get("last_shot_outcome")
286                 ))
287             elif sport == "tabletennis":
288                 parsed_items.append(TableTennisRally(
289                     video_id=video_id,
290                     rally_number=_safe_int(row.get("rally_number"), idx + 1),
291                     start_time=_safe_float(row.get("start_time")),
292                     end_time=_safe_float(row.get("end_time")),

```



```

293             score_before=row.get("score_before"),
294             score_after=row.get("score_after"),
295             shots_count=_safe_int(row.get("shots_count"), None),
296             ending_reason=row.get("ending_reason"),
297             last_shot_outcome=row.get("last_shot_outcome")
298         ))
299     else:
300         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
301
302     return parsed_items
303
304 def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
305     """Download one video at best resolution, retrying transient failures.
306
307     A below-min-height result almost always means YouTube rate-limited the
308     request and served a low fallback (not that the video lacks HD), so we
309     retry with a growing backoff. Returns (DownloadResult, None) on success,
310     or (None, failure_reason) once retries are exhausted. A below-min file is
311     discarded each attempt so it never lands in the DB.
312     """
313     attempts = max(1, args.retries + 1)
314     reason = "unknown error"
315     for i in range(attempts):
316         result = download_best_video(
317             url, dest_path,
318             max_height=max_height, max_size_mb=max_size_mb,
319             cookies_from_browser=args.cookies_from_browser,
320             cookies_file=args.cookies,
321             player_client=args.player_client,
322         )
323         if result.success and (result.height is None or result.height >=
args.min_height):
324             return result, None
325
326         if not result.success:
327             reason = result.error or "unknown error"
328         else:
329             reason = (f"got {result.resolution_label} (< min-height
{args.min_height})p) ? "
330                     f"YouTube served a low format (rate-limited?)")
331         try:
332             os.remove(result.path)
333         except OSError:
334             pass
335
336         if i < attempts - 1:
337             backoff = args.retry_backoff * (i + 1)
338             print(f" attempt {i+1}/{attempts} failed: {reason}\n retrying in
{backoff:.0f}s...")
339             time.sleep(backoff)
340     return None, reason
341
342 def fetch(self, args):
343     """(Re)download videos at best available resolution, updating the DB in place.
344
345     This is the in-place upgrade path: rows, rally annotations, and curation
346     status are all preserved ? only the video file plus its `local_path`,
347     `resolution`, and `fps` are refreshed. By default it skips videos that
348     already have a local file at or above `--min-height`, so re-runs are cheap
349     and resumable; `--force` re-fetches everything.
350     """
351     storage = self.config.get("video_storage", {})
352     local_dir = storage.get("local_dir", "./data/videos")
353     max_download_size_mb = storage.get("max_download_size_mb")

```

```

354         max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
355         os.makedirs(local_dir, exist_ok=True)
356
357         commercial_only = not args.include_noncommercial
358         if args.status == "all":
359             statuses = [CurationStatus.CURATED, CurationStatus.STAGING]
360         else:
361             statuses = [CurationStatus(args.status)]
362
363         supported = self.config.get("supported_sports", ["badminton"])
364         sport_strs = [args.sport] if args.sport else supported
365
366         # Collect target videos.
367         videos = []
368         for sport_str in sport_strs:
369             try:
370                 sport = SportType(sport_str.lower())
371             except ValueError:
372                 print(f"Warning: skipping unknown sport '{sport_str}'.")
373                 continue
374             for status in statuses:
375                 videos.extend(self.db.get_videos_by_status(status, sport))
376         if args.video_id:
377             videos = [v for v in videos if v.video_id == args.video_id]
378
379         if not videos:
380             print("\nNo matching videos to fetch.\n")
381             return
382
383         results = [] # (video_id, action, detail)
384         for video in videos:
385             if commercial_only and not video.is_commercial:
386                 results.append((video.video_id, "skip", "non-commercial"))
387                 continue
388             if not video.video_url or "watch?v=example" in video.video_url:
389                 # Either no URL, or the parser's placeholder (the ShuttleSet catalog
390                 # had no URL for this match) ? nothing to download.
391                 results.append((video.video_id, "skip", "no real source URL"))
392                 continue
393
394             dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
395
396             # Skip if we already have a good-enough local file (unless --force).
397             if not args.force and os.path.exists(dest_path):
398                 _, height, _ = probe_resolution_fps(dest_path)
399                 if height is not None and height >= args.min_height:
400                     results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
401                     continue
402
403             print(f"Fetching {video.video_id} ({video.match_name})...")
404             result, fail_reason = self._fetch_one_with_retry(
405                 video.video_url, dest_path, args, max_height, max_download_size_mb)
406             if fail_reason is not None:
407                 results.append((video.video_id, "FAIL", fail_reason))
408                 if args.sleep:
409                     time.sleep(args.sleep)
410                 continue
411
412             # Update only the file-derived fields; preserve everything else (UPSERT
413             # mutates the row in place, leaving rallies intact).
414             video.local_path = result.path
415             if result.resolution_label:
416                 video.resolution = result.resolution_label

```

```

417         if result.fps:
418             video.fps = result.fps
419         self.db.insert_video(video)
420
421         detail = (f"{result.resolution_label or '?}'"
422                 f"{f' {result.fps:g}fps' if result.fps else ''},
423 {result.size_mb:.0f} MB")
424         results.append((video.video_id, "fetched", detail))
425         if args.sleep:
426             time.sleep(args.sleep)
427
428         print("\n=== Fetch summary ===")
429         print(tabulate([[vid, action, detail] for vid, action, detail in results],
430                       headers=["Video ID", "Action", "Detail"], tablefmt="grid"))
431         fetched = sum(1 for _, a, _ in results if a == "fetched")
432         failed = [r for r in results if r[1] == "FAIL"]
433         print(f"\nFetched/updated : {fetched}    Already-good : {sum(1 for _,a,_ in
434 results if a=='have'))}    "
435               f"Skipped : {sum(1 for _,a,_ in results if a=='skip')}    Failed :
436 {len(failed)}")
437         if failed:
438             print("Failures (left unchanged in DB):")
439             for vid, _, detail in failed:
440                 print(f" - {vid}: {detail}")
441             print()
442
443     def export_eval_set(self, args):
444         """Export curated annotations as indexer-ready eval/training sets.
445
446         Emits, per qualifying video, a `//.csv` in the
447         indexer's `start,end` ground-truth format (decimal seconds), plus a single
448         `/manifest.json` that pairs each CSV with its video file. The
449         manifest is the contract the downstream eval/training driver reads: the
450         indexer keys videos by `*file MD5*`, not by our `video_id`, so it must run
451         against the exact `local_path`/`video_url` recorded here.
452
453         By default only CURATED + commercially-safe videos are exported, mirroring
454         the license gate already enforced elsewhere in the toolchain.
455         """
456         out_dir = args.out_dir
457         commercial_only = not args.include_noncommercial
458         statuses = [CurationStatus.CURATED]
459         if args.include_staging:
460             statuses.append(CurationStatus.STAGING)
461
462         supported = self.config.get("supported_sports", ["badminton"])
463         sport_strs = [args.sport] if args.sport else supported
464
465         manifest = []
466         skipped_noncomm = 0
467         skipped_empty = 0
468
469         for sport_str in sport_strs:
470             try:
471                 sport = SportType(sport_str.lower())
472             except ValueError:
473                 print(f"Warning: skipping unknown sport '{sport_str}'.")
474                 continue
475
476             videos = []
477             for status in statuses:
478                 videos.extend(self.db.get_videos_by_status(status, sport))
479
480             for video in videos:
481                 if commercial_only and not video.is_commercial:

```

```

479         skipped_noncomm += 1
480         continue
481
482     intervals = self.db.get_segment_intervals(video.video_id, sport)
483     if not intervals:
484         skipped_empty += 1
485         continue
486
487     sport_dir = os.path.join(out_dir, sport.value)
488     os.makedirs(sport_dir, exist_ok=True)
489     csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
490     with open(csv_path, "w", newline="", encoding="utf-8") as f:
491         writer = csv.writer(f)
492         writer.writerow(["start", "end"])
493         for start, end in intervals:
494             writer.writerow([start, end])
495
496     manifest.append({
497         "video_id": video.video_id,
498         "sport": sport.value,
499         "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
500         "local_path": video.local_path,
501         "video_url": video.video_url,
502         "match_name": video.match_name,
503         "fps": video.fps,
504         "resolution": video.resolution,
505         "license_type": video.license_type.value,
506         "is_commercial": video.is_commercial,
507         "status": video.status.value,
508         "num_segments": len(intervals),
509     })
510
511     os.makedirs(out_dir, exist_ok=True)
512     manifest_doc = {
513         "commercial_only": commercial_only,
514         "statuses": [s.value for s in statuses],
515         "total_videos": len(manifest),
516         "total_segments": sum(m["num_segments"] for m in manifest),
517         "eval_sets": manifest,
518     }
519     manifest_path = os.path.join(out_dir, "manifest.json")
520     with open(manifest_path, "w", encoding="utf-8") as f:
521         json.dump(manifest_doc, f, indent=2)
522
523     print(f"\n=== Exported eval/training set to '{out_dir}' ===")
524     if manifest:
525         summary = [[m["sport"], m["video_id"], m["num_segments"],
526                     "YES" if m["is_commercial"] else "NO",
527                     os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
528                     for m in manifest]
529         print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
530         print(f"\nVideos exported : {len(manifest)}")
531         print(f"Total segments : {manifest_doc['total_segments']}")
532         if skipped_noncomm:
533             print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
534         if skipped_empty:
535             print(f"Skipped (no annotations) : {skipped_empty}")
536         print(f"Manifest : {manifest_path}\n")
537
538     def convert_cvat(self, args):
539         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
540

```

```

541     Absorbs the CVAT bounding-box delivery format (rally fields carried as box
542     attributes) on the ingestion side: each rally's time is recomputed from
543     its frame range at the video's true fps, and content defects (overlaps,
544     zero-duration, off-vocabulary endings, coarse sampling) are written to an
545     issues report rather than silently "fixed". The emitted CSV imports via
546     `import-local`; any blocker issue must be resolved first, since the
547     importer's validator rejects overlapping / zero-duration rallies.
548     """
549     from src.parsers.cvat.rally_cvat import (
550         convert, write_canonical_csv, render_issue_report,
551     )
552     try:
553         result = convert(args.input, fps=args.fps,
554                         video_path=args.video_path, step=args.step)
555     except (FileNotFoundError, ValueError) as e:
556         print(f"Error: {e}")
557         sys.exit(1)
558
559     write_canonical_csv(result.records, args.out)
560
561     # Observability report next to the canonical CSV. Never raises.
562     from src.reporting import emit_delivery_report
563     src_name = os.path.splitext(os.path.basename(args.input))[0]
564     dur = (result.meta.get("stop_frame") / result.fps) if
565     (result.meta.get("stop_frame") and result.fps) else None
566     emit_delivery_report(
567         out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
568         records=result.records, issues=result.issues, fps=result.fps,
569         video_path=args.video_path,
570         video_duration_s=dur, source_format="cvat", step=result.step,
571     )
572
573     report = render_issue_report(result, os.path.basename(args.input))
574     if args.issues:
575         with open(args.issues, "w", encoding="utf-8") as f:
576             f.write(report)
577     print(report)
578     print(f"Canonical CSV written: {args.out} ({len(result.records)} rallies)")
579     if result.n_blockers:
580         print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject
581         "
582         f"this file until they are resolved (by the vendor or manual
583         review).")
584     else:
585         print("\n No blockers. Next:")
586         print(f"    python -m src.cli.curate import-local --sport <sport> "
587               f"--video-path <video> --annotations-path {args.out} --fps
588               {result.fps:g}")
589
590     def validate_delivery(self, args):
591         """Run the full validation suite on a rally delivery and emit a report.
592
593         Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
594         CSV (--csv). Writes, into --out-dir:
595             <id>_report.md    human report (scoreboard + issues + rallies to review)
596             <id>_report.json  machine summary (for CI / a bulk-ingest loop)
597             <id>_review.csv   the annotatable manual-verification sheet
598         Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
599         """
600         import json
601         from src.parsers.cvat.rally_cvat import (
602             convert, load_canonical_csv, detect_issues,
603             build_summary, render_validation_md, write_review_csv,
604         )
605         vid = args.video_id or "delivery"

```

```

601         fps = None
602         step = None
603         stop_frame = None
604         if args.input:
605             try:
606                 result = convert(args.input, fps=args.fps, video_path=args.video_path)
607             except (FileNotFoundError, ValueError) as e:
608                 print(f"Error: {e}")
609                 sys.exit(1)
610             records, issues, fps = result.records, result.issues, result.fps
611             step = result.step
612             stop_frame = result.meta.get("stop_frame")
613         elif args.csv:
614             try:
615                 records = load_canonical_csv(args.csv)
616             except FileNotFoundError as e:
617                 print(f"Error: {e}")
618                 sys.exit(1)
619             if not records:
620                 print(f"Error: no usable rally rows in {args.csv}")
621                 sys.exit(1)
622             issues = detect_issues(records)
623         else:
624             print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
625             sys.exit(1)
626
627         os.makedirs(args.out_dir, exist_ok=True)
628         summary = build_summary(records, issues, source=vid, fps=fps)
629         md = render_validation_md(records, issues, source=vid)
630         json_path = os.path.join(args.out_dir, f"{vid}_report.json")
631         md_path = os.path.join(args.out_dir, f"{vid}_report.md")
632         review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
633         with open(json_path, "w", encoding="utf-8") as f:
634             json.dump(summary, f, indent=2)
635         with open(md_path, "w", encoding="utf-8") as f:
636             f.write(md)
637         write_review_csv(records, issues, review_path)
638
639         # Observability report (envelope JSON + technical md + customer summary),
640         # colocated with the existing artifacts. Never raises.
641         from src.reporting import emit_delivery_report
642         duration_s = (stop_frame / fps) if (stop_frame and fps) else None
643         report_paths = emit_delivery_report(
644             out_dir=args.out_dir, source=vid, records=records, issues=issues, fps=fps,
645             video_path=args.video_path, video_duration_s=duration_s,
646             source_format=("cvat" if args.input else "csv"), step=step,
647         )
648
649         print(md)
650         print(f"\nWrote:\n {md_path}\n {json_path}\n {review_path}")
651         if report_paths:
652             print(f" {report_paths.get('technical_md')}\n
653 {report_paths.get('customer_md')}\n {report_paths.get('json')}")
654             if summary["n_blockers"]:
655                 print(f"\n GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
656 resolved.")
657                 sys.exit(2)
658             print("\n GATE: PASS ? review the flagged rallies, then `import-local`.")
659
660     def curate(self):
661         """Interactive loop to review and curate staging video annotations."""
662         staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
663         if not staging_videos:
664             print("\nNo staging video annotations to review.\n")
665             return

```

```

664
665     print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
666     table_data = []
667     for idx, video in enumerate(staging_videos):
668         table_data.append([
669             idx + 1,
670             video.video_id,
671             video.match_name,
672             video.sport.value,
673             video.license_type.value,
674             "YES" if video.is_commercial else "NO"
675         ])
676
677     print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport",
678 "License", "Commercial Safe?"], tablefmt="grid"))
679     print("\nEnter a number (1-{}) to review details, or 'q' to
680 quit:".format(len(staging_videos)))
681
682     choice = input("> ").strip()
683     if choice.lower() == 'q':
684         return
685
686     try:
687         idx = int(choice) - 1
688         if idx < 0 or idx >= len(staging_videos):
689             print("Invalid choice.")
690             return
691         self._curate_video_interactive(staging_videos[idx])
692     except ValueError:
693         print("Invalid input.")
694
695 def _curate_video_interactive(self, video: VideoMetadata):
696     """Displays details of a specific staging video and prompts for curation
697     action."""
698
699     # Load and count events
700     rallies = []
701     pts = []
702     delivs = []
703     if video.sport == SportType.BADMINTON:
704         rallies = self.db.get_badminton_rallies(video.video_id)
705     elif video.sport == SportType.PICKLEBALL:
706         rallies = self.db.get_pickleball_rallies(video.video_id)
707     elif video.sport == SportType.TABLE_TENNIS:
708         rallies = self.db.get_tabletennis_rallies(video.video_id)
709     elif video.sport == SportType.TENNIS:
710         pts = self.db.get_tennis_points(video.video_id)
711     elif video.sport == SportType.CRICKET:
712         delivs = self.db.get_cricket_deliveries(video.video_id)
713
714     while True:
715         print("\n===== REVIEWING VIDEO =====")
716         print(f"Match Name      : {video.match_name}")
717         print(f"Video ID       : {video.video_id}")
718         print(f"Sport         : {video.sport.value}")
719         print(f"Video URL      : {video.video_url or 'N/A'}")
720         print(f"Local Path     : {video.local_path or 'N/A'}")
721         print(f"License        : {video.license_type.value} ({video.license_url or 'no
722 URL'})")
723
724         print(f"Commercial OK: {video.is_commercial}")
725
726         if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
727 SportType.TABLE_TENNIS):
728             print(f"Total Rallies: {len(rallies)}")
729             if rallies:

```

```

724         print("Sample rallies:")
725         sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
r.score_after, r.shots_count] for r in rallies[:5]]
726         print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
"Shots"], tablefmt="simple"))
727     elif video.sport == SportType.TENNIS:
728         print(f"Total Points : {len(pts)}")
729         if pts:
730             print("Sample points:")
731             sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
p.game_score] for p in pts[:5]]
732             print(tabulate(sample_data, headers=["Point #", "Duration",
"Score"], tablefmt="simple"))
733         elif video.sport == SportType.CRICKET:
734             print(f"Total Balls : {len(delivs)}")
735             if delivs:
736                 print("Sample deliveries:")
737                 sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
738                 print(tabulate(sample_data, headers=["Over.Ball", "Duration",
"Runs", "Wicket"], tablefmt="simple"))
739
740         print("\nChoose Curation Action:")
741         print(" [a] Approve (promote to CURATED)")
742         print(" [r] Reject (delete from database)")
743         print(" [p] Play a rally/point segment in browser")
744         print(" [k] Keep in Staging (do nothing and go back)")
745
746         action = input("> ").strip().lower()
747         if action == 'a':
748             self.db.update_status(video.video_id, CurationStatus.CURATED)
749             print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
750             break
751         elif action == 'r':
752             self.db.delete_video(video.video_id)
753             print(f"Successfully rejected and deleted '{video.match_name}'.\n")
754             break
755         elif action == 'k':
756             print("Curation state unchanged.\n")
757             break
758         elif action == 'p':
759             self._play_segment_interactive(video, rallies, pts, delivs)
760         else:
761             print("Invalid action selected.")
762
763     def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
764         import webbrowser
765
766         # Get start/end based on user input
767         start_time = None
768         end_time = None
769         label = "segment"
770
771         if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
772             if not rallies:
773                 print("No rallies to play.")
774                 return
775             idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
776             try:
777                 rally_num = int(idx_str)
778                 rally = next((r for r in rallies if r.rally_number == rally_num), None)
779                 if rally:
780                     start_time = rally.start_time

```



```

781         end_time = rally.end_time
782         label = f"Rally {rally_num}"
783     except ValueError:
784         pass
785 elif video.sport == SportType.TENNIS:
786     if not pts:
787         print("No points to play.")
788         return
789     idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
790     try:
791         pt_num = int(idx_str)
792         pt = next((p for p in pts if p.point_number == pt_num), None)
793         if pt:
794             start_time = pt.start_time
795             end_time = pt.end_time
796             label = f"Point {pt_num}"
797     except ValueError:
798         pass
799 elif video.sport == SportType.CRICKET:
800     if not delivs:
801         print("No deliveries to play.")
802         return
803     over_str = input("Enter Over number: ").strip()
804     ball_str = input("Enter Ball number: ").strip()
805     try:
806         o_num = int(over_str)
807         b_num = int(ball_str)
808         d = next((x for x in delivs if x.over == o_num and x.ball == b_num),
809 None)
810         if d:
811             start_time = d.start_time
812             end_time = d.end_time
813             label = f"Over {o_num} Ball {b_num}"
814     except ValueError:
815         pass
816 if start_time is None or end_time is None:
817     print("Segment not found or invalid input.")
818     return
819
820 print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")
821
822 # Check video location
823 play_url = None
824 if video.local_path and os.path.exists(video.local_path):
825     abs_path = os.path.abspath(video.local_path)
826     # Create file:// URL with media fragment (browsers support #t=start,end
827 fragment)
828     play_url = f"file:/// {abs_path.replace(os.sep,
829 '/' )}#t={start_time},{end_time}"
830 elif video.video_url:
831     # YouTube URL: seek to start time
832     if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
833         t_sec = int(start_time)
834         sep = "&" if "?" in video.video_url else "?"
835         play_url = f"{video.video_url}{sep}t={t_sec}s"
836     else:
837         play_url = video.video_url
838
839 if play_url:
840     print(f"Opening browser: {play_url}")
841     webbrowser.open(play_url)
842 else:
843     print("Error: No local video file or remote URL available to play.")

```

```

843
844     def main():
845         parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
846         subparsers = parser.add_subparsers(dest="command")
847
848         # Status command
849         subparsers.add_parser("status", help="Display summary status of sports metadata
database")
850
851         # Curate command
852         subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")
853
854         # Import-local command
855         import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
856         import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
badminton, tennis, cricket)")
857         import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
858         import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
859         import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
860         import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
861         import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
862         import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per
second of the video")
863         import_parser.add_argument("--resolution", default="1080p", help="Resolution of the
video (e.g., 1080p, 720p)")
864
865         # Fetch command ? (re)download videos at best resolution, update DB in place
866         fetch_parser = subparsers.add_parser(
867             "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
868         fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
869         fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
870         fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
871                                     help="Which curation statuses to fetch (default: all)")
872         fetch_parser.add_argument("--min-height", type=int, default=720,
873                                     help="Skip videos already at/above this height unless
--force (default: 720)")
874         fetch_parser.add_argument("--max-height", type=int, default=None,
875                                     help="Cap download resolution (e.g. 1080); default: best
available")
876         fetch_parser.add_argument("--force", action="store_true",
877                                     help="Re-download even if a good-enough local file already
exists")
878         fetch_parser.add_argument("--include-noncommercial", action="store_true",
879                                     help="Include videos whose license is not commercially
safe (default: commercial only)")
880         fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",
metavar="BROWSER",
881                                     help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
882                                     "A Premium login gives reliable best-resolution
downloads. "
883                                     "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
884         fetch_parser.add_argument("--cookies", metavar="FILE",
885                                     help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")

```

```

886         fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
887                                     help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
888                                     "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.")
889         fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
890                                     help="Pause between videos to avoid YouTube rate-limiting
(default: 4)")
891         fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
892                                     help="Retries per video when a download fails or returns a
low format (default: 2)")
893         fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
894                                     help="Base backoff between retries; grows linearly per
attempt (default: 20)")
895
896         # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
897         cvat_parser = subparsers.add_parser(
898             "convert-cvat",
899             help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
900         cvat_parser.add_argument("--input", required=True,
901                                     help="Path to the CVAT XML export (image- or track-mode)")
902         cvat_parser.add_argument("--out", required=True,
903                                     help="Path to write the canonical per-rally CSV (import-
local input)")
904         cvat_parser.add_argument("--video-path",
905                                     help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
906         cvat_parser.add_argument("--fps", type=float,
907                                     help="Override fps instead of probing the video")
908         cvat_parser.add_argument("--step", type=int,
909                                     help="Frame sampling step (default: read from CVAT meta,
else inferred)")
910         cvat_parser.add_argument("--issues",
911                                     help="Path to also write the human-readable issues report")
912
913         # Validate-delivery command ? run the full check suite + emit a structured report
914         vd_parser = subparsers.add_parser(
915             "validate-delivery",
916             help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
917         vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
918         vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
919         vd_parser.add_argument("--video-path",
920                                     help="Source video; fps via ffprobe (when using --input)")
921         vd_parser.add_argument("--fps", type=float, help="Override fps (when using
--input)")
922         vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
923         vd_parser.add_argument("--out-dir", default="data/validation",
924                                     help="Where to write the report + review sheet")
925
926         # Export command
927         export_parser = subparsers.add_parser(
928             "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
929         export_parser.add_argument("--out-dir", default="data/eval_export",
930                                     help="Output directory for per-video CSVs and
manifest.json")
931         export_parser.add_argument("--sport",
932                                     help="Restrict to a single sport (default: all

```

```

supported_sports)")
933     export_parser.add_argument("--include-staging", action="store_true",
934                               help="Also export videos still in staging (default:
curated only)")
935     export_parser.add_argument("--include-noncommercial", action="store_true",
936                               help="Include videos whose license is not commercially
safe (default: commercial only)")
937
938     args = parser.parse_args()
939     cli = SportsDataCLI()
940
941     if args.command == "status":
942         cli.status()
943     elif args.command == "curate":
944         cli.curate()
945     elif args.command == "import-local":
946         cli.import_local(args)
947     elif args.command == "convert-cvat":
948         cli.convert_cvat(args)
949     elif args.command == "validate-delivery":
950         cli.validate_delivery(args)
951     elif args.command == "export":
952         cli.export_eval_set(args)
953     elif args.command == "fetch":
954         cli.fetch(args)
955     else:
956         # Default: print help
957         parser.print_help()
958
959 if __name__ == "__main__":
960     main()
961

```

16. assistant (2026-06-08T08:39:14.345Z)

Now let me search for the schema definitions for rally types and check if there are any annotation/provenance tables:

17. user (2026-06-08T08:39:14.755Z)

```

1     from pydantic import BaseModel, Field, model_validator
2     from typing import Optional
3
4     class BadmintonRally(BaseModel):
5         video_id: str
6         rally_number: int = Field(..., description="Chronological rally index in the match")
7         start_time: float = Field(..., description="Start of rally in seconds")
8         end_time: float = Field(..., description="End of rally in seconds")
9         score_before: Optional[str] = Field(None, description="Score before the rally (e.g.
'0-0')")
10        score_after: Optional[str] = Field(None, description="Score after the rally (e.g.
'1-0')")
11        shots_count: Optional[int] = Field(None, description="Number of strokes in the
rally")
12        ending_reason: Optional[str] = Field(None, description="Why the rally ended:
'winner' | 'forced_error' | 'unforced_error' | 'service_fault' | 'let' | 'other'")
13        last_shot_outcome: Optional[str] = Field(None, description="Where the last shot
landed: 'in' | 'net' | 'out' | 'n_a' (n_a for service_fault/let)")
14
15        @model_validator(mode='after')
16        def validate_timestamps(self) -> 'BadmintonRally':
17            if self.start_time < 0:
18                raise ValueError("start_time must be non-negative")
19            if self.end_time <= self.start_time:
20                raise ValueError("end_time must be strictly greater than start_time")

```

```
21         return self
22
```

18. user (2026-06-08T08:39:14.974Z)

```
1     from pydantic import BaseModel, Field, model_validator
2     from typing import Optional
3
4     class PickleballRally(BaseModel):
5         video_id: str
6         rally_number: int = Field(..., description="Chronological rally index in the match")
7         start_time: float = Field(..., description="Start of rally in seconds (serve
contact)")
8         end_time: float = Field(..., description="End of rally in seconds (ball dead)")
9         score_before: Optional[str] = Field(None, description="Score before the rally (e.g.
'0-0')")
10        score_after: Optional[str] = Field(None, description="Score after the rally (e.g.
'1-0')")
11        shots_count: Optional[int] = Field(None, description="Number of paddle-ball contacts
in the rally (serve = shot 1; bounces are not shots)")
12        ending_reason: Optional[str] = Field(None, description="Why the rally ended:
'winner' | 'forced_error' | 'unforced_error' | 'service_fault' | 'let' | 'other'")
13        last_shot_outcome: Optional[str] = Field(None, description="Where the last shot
landed: 'in' | 'net' | 'out' | 'n_a' (n_a for service_fault/let)")
14
15        @model_validator(mode='after')
16        def validate_timestamps(self) -> 'PickleballRally':
17            if self.start_time < 0:
18                raise ValueError("start_time must be non-negative")
19            if self.end_time <= self.start_time:
20                raise ValueError("end_time must be strictly greater than start_time")
21            return self
22
```

19. user (2026-06-08T08:39:15.492Z)

C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:# here plus its CREATE TABLE, with no new query code.

C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:	CREATE TABLE IF NOT EXISTS videos (
C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:	CREATE TABLE IF NOT EXISTS badminton_rallies (
C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:	CREATE TABLE IF NOT EXISTS tennis_points (
C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:	CREATE TABLE IF NOT EXISTS cricket_deliveries (
C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:	CREATE TABLE IF NOT EXISTS pickleball_rallies (
C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:	CREATE TABLE IF NOT EXISTS tabletennis_rallies (
C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:	# column. CREATE TABLE IF NOT EXISTS won't alter an existing table, so

20. assistant (2026-06-08T08:39:17.750Z)

Let me search for any audit/annotation audit tracking or maturity fields:

21. user (2026-06-08T08:39:18.563Z)

```
C:/Users/avidu/Projects/sports-data-collector/cleanup.py
C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py
C:/Users/avidu/Projects/sports-data-collector/src/core/downloader.py
C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py
C:/Users/avidu/Projects/sports-data-collector/src/reporting/adapter.py
```

```
C:/Users/avidu/Projects/sports-data-collector/tests/test_cli.py
C:/Users/avidu/Projects/sports-data-collector/tests/test_cvat_parser.py
C:/Users/avidu/Projects/sports-data-collector/tests/test_downloader.py
C:/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py
```

22. assistant (2026-06-08T08:39:23.070Z)

Let me look at the validator and the CVAT parser for more details:

23. user (2026-06-08T08:39:23.602Z)

```
C:/Users/avidu/Projects/sports-data-collector/src/core/config.py
C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py
```

24. user (2026-06-08T08:39:25.701Z)

```
1      from typing import List, Tuple, Union
2      from src.core.config import load_config
3      from src.core.schemas.base import VideoMetadata, LicenseType
4      from src.core.schemas.badminton import BadmintonRally
5      from src.core.schemas.tennis import TennisPoint
6      from src.core.schemas.cricket import CricketDelivery
7      from src.core.schemas.pickleball import PickleballRally
8      from src.core.schemas.tabletennis import TableTennisRally
9
10     class DatasetValidator:
11         def __init__(self, config_path: str = "config/config.yaml"):
12             self.config_path = config_path
13             self.commercial_licenses = self._load_licenses_whitelist()
14
15         def _load_licenses_whitelist(self) -> List[str]:
16             # load_config returns {} on a missing/unreadable file, so the default list
17             # below is the single fallback (keep in sync with config.yaml).
18             config = load_config(self.config_path)
19             return config.get(
20                 'commercial_licenses',
21                 ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
22                 "Proprietary"],
23             )
24
25         def check_license_commercial(self, license_type: Union[LicenseType, str]) -> bool:
26             """Determines if a license type is safe for commercial use based on the
27             configuration whitelist."""
28             val = license_type.value if isinstance(license_type, LicenseType) else
29             license_type
30             return val in self.commercial_licenses
31
32         def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
33             List[str]]:
34             """
35             Validates badminton rallies:
36             - Sorted chronologically.
37             - Start time < End time (handled by Pydantic, but double checked).
38             - No temporal overlap between sequential rallies.
39             """
40             errors = []
41             if not rallies:
42                 return True, errors
43
44             # Sort by rally number to check progression
45             sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
46
47             for i in range(len(sorted_rallies)):
48                 r = sorted_rallies[i]
```

```

45         if r.start_time >= r.end_time:
46             errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
47
48         if i > 0:
49             prev = sorted_rallies[i - 1]
50             if r.start_time < prev.end_time:
51                 errors.append(
52                     f"Rally overlap detected between Rally {prev.rally_number} "
53                     f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
54                     f"(starts at {r.start_time}s)"
55                 )
56
57         return len(errors) == 0, errors
58
59     def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool,
List[str]]:
60         errors = []
61         if not points:
62             return True, errors
63
64         sorted_pts = sorted(points, key=lambda p: p.point_number)
65         for i in range(len(sorted_pts)):
66             pt = sorted_pts[i]
67             if pt.start_time >= pt.end_time:
68                 errors.append(f"Point {pt.point_number}: start ({pt.start_time}) >= end
({pt.end_time})")
69
70             if i > 0:
71                 prev = sorted_pts[i - 1]
72                 if pt.start_time < prev.end_time:
73                     errors.append(
74                         f"Point overlap detected: Point {prev.point_number} (ends
{prev.end_time}s) "
75                         f"and Point {pt.point_number} (starts {pt.start_time}s)"
76                     )
77         return len(errors) == 0, errors
78
79     def validate_cricket_deliveries(self, deliveries: List[CricketDelivery]) ->
Tuple[bool, List[str]]:
80         errors = []
81         if not deliveries:
82             return True, errors
83
84         # Sort by over, then ball within over
85         sorted_delivs = sorted(deliveries, key=lambda d: (d.over, d.ball))
86         for i in range(len(sorted_delivs)):
87             d = sorted_delivs[i]
88             if d.start_time >= d.end_time:
89                 errors.append(f"Over {d.over} Ball {d.ball}: start ({d.start_time}) >=
end ({d.end_time})")
90
91             if i > 0:
92                 prev = sorted_delivs[i - 1]
93                 if d.start_time < prev.end_time:
94                     errors.append(
95                         f"Delivery overlap: Over {prev.over} Ball {prev.ball} (ends
{prev.end_time}s) "
96                         f"and Over {d.over} Ball {d.ball} (starts {d.start_time}s)"
97                     )
98         return len(errors) == 0, errors
99
100     def validate_pickleball_rallies(self, rallies: List[PickleballRally]) -> Tuple[bool,
List[str]]:
101         """Validates pickleball rallies: positive durations and no temporal overlap

```

```

102         between chronologically-ordered rallies (same contract as badminton)."""
103         errors = []
104         if not rallies:
105             return True, errors
106
107         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
108         for i in range(len(sorted_rallies)):
109             r = sorted_rallies[i]
110             if r.start_time >= r.end_time:
111                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
112
113             if i > 0:
114                 prev = sorted_rallies[i - 1]
115                 if r.start_time < prev.end_time:
116                     errors.append(
117                         f"Rally overlap detected between Rally {prev.rally_number} "
118                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
119                         f"(starts at {r.start_time}s)"
120                     )
121         return len(errors) == 0, errors
122
123     def validate_tabletennis_rallies(self, rallies: List[TableTennisRally]) ->
Tuple[bool, List[str]]:
124         """Validates table-tennis rallies: positive durations and no temporal overlap
125         between chronologically-ordered rallies (same contract as badminton)."""
126         errors = []
127         if not rallies:
128             return True, errors
129
130         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
131         for i in range(len(sorted_rallies)):
132             r = sorted_rallies[i]
133             if r.start_time >= r.end_time:
134                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
135
136             if i > 0:
137                 prev = sorted_rallies[i - 1]
138                 if r.start_time < prev.end_time:
139                     errors.append(
140                         f"Rally overlap detected between Rally {prev.rally_number} "
141                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
142                         f"(starts at {r.start_time}s)"
143                     )
144         return len(errors) == 0, errors
145

```

25. user (2026-06-08T08:39:25.776Z)

```

1     """Central configuration loader.
2
3     A single place to read ``config/config.yaml`` so modules don't each re-open the
4     file with slightly different fallbacks. Returns ``{}`` when the file is missing or
5     unreadable (callers apply their own defaults), matching the prior per-module
6     behaviour but without the duplicated boilerplate and broad excepts.
7     """
8     import os
9     from typing import Any, Dict
10
11     import yaml
12
13     DEFAULT_CONFIG_PATH = "config/config.yaml"
14     DEFAULT_DB_PATH = "data/sports_metadata.db"
15

```



```

16
17 def load_config(config_path: str = DEFAULT_CONFIG_PATH) -> Dict[str, Any]:
18     """Load a YAML config into a dict; return {} if missing/unreadable/empty."""
19     if not os.path.exists(config_path):
20         return {}
21     try:
22         with open(config_path, "r", encoding="utf-8") as f:
23             return yaml.safe_load(f) or {}
24     except (OSError, yaml.YAMLError):
25         return {}
26
27
28 def get_database_path(config: Dict[str, Any]) -> str:
29     """Resolve the SQLite database path from a loaded config, with a default."""
30     return config.get("database_path", DEFAULT_DB_PATH)
31

```

26. assistant (2026-06-08T08:39:27.833Z)

Now let me check the testing files to understand what the observability report captures:

27. user (2026-06-08T08:39:28.083Z)

```

1     """Tests for the collector observability reports. obsreport is a soft dependency;
2     skip cleanly if it isn't installed."""
3     import os
4     from types import SimpleNamespace
5
6     import pytest
7
8     pytest.importorskip("obsreport")
9
10    from src.parsers.cvat.rally_cvat import Issue, RallyRecord # noqa: E402
11    from src.reporting import emit_delivery_report, emit_import_report # noqa: E402
12    from src.reporting import adapter # noqa: E402
13
14
15    def _rec(n, start, end, shots=5):
16        return RallyRecord(rally_number=n, start_time=start, end_time=end,
shots_count=shots,
17                           ending_reason="winner", last_shot_outcome="in",
score_before=None,
18                           score_after=None, notes=None, frame_min=int(start * 60),
frame_max=int(end * 60), n_boxes=3)
19
20
21    def _codes(env):
22        return {f.code for f in env.flags}
23
24
25    # --- issue -> flag -----
26    def test_issue_to_flag_severity_and_customer_message():
27        blocker = adapter.issue_to_flag(Issue("blocker", "overlap", "Rally 1 overlaps Rally
2.", 2))
28        assert blocker.severity == "error"
29        assert blocker.customer_message == "Rally 1 overlaps Rally 2." # actionable ->
shown to vendor
30        assert blocker.faq_ref.endswith("#what-does-validation-check")
31        assert blocker.evidence == {"rally_number": 2}
32
33        low = adapter.issue_to_flag(Issue("low", "outcome_vocab", "odd outcome", 3))
34        assert low.severity == "info"
35        assert low.customer_message is None # cosmetic -> not surfaced to the vendor
36
37

```

```

38 # --- delivery envelope -----
39 def test_delivery_with_blocker_fails_and_surfaces_to_customer():
40     records = [_rec(1, 0.0, 5.0), _rec(2, 4.0, 9.0)]
41     issues = [Issue("blocker", "overlap", "Rally 1 (ends 5.00s) overlaps Rally 2 (starts
4.00s).", 2)]
42     rec = adapter.build_delivery_recorder(records=records, issues=issues, fps=59.94,
43                                           source="pilot_01", video_duration_s=600.0)
44     env = rec.envelope
45     assert env.verdict == "fail"
46     assert env.get_stage("validation").status == "error"
47     assert "overlap" in _codes(env)
48     assert env.highlights.segment_count == 2
49     assert env.lint() == []
50
51     from obsreport import render_customer, render_technical
52     tech = render_technical(env, rec.spec)
53     cust = render_customer(env, rec.spec)
54     assert "***overlap***" in tech # bolded issue CODE visible technically
55     assert "***overlap***" not in cust # code hidden from the customer/vendor view
56     assert "overlaps Rally 2" in cust # but the plain message is shown
57     assert "blocking issues" in cust # collector-specific customer verdict (not the
generic default)
58
59
60 def test_delivery_flag_carries_frame_evidence_and_review_framing():
61     recs = [_rec(1, 2.0, 7.0), _rec(2, 10.0, 14.0), _rec(3, 40.0, 44.0)]
62     issues = [
63         Issue("blocker", "overlap", "Rally 1 (ends 7.00s) overlaps Rally 2 (starts
10.00s).", 2),
64         Issue("medium", "possible_missed_rally", "26.0s unlabeled gap between Rally 2
and Rally 3.", 3),
65     ]
66     rec = adapter.build_delivery_recorder(records=recs, issues=issues, fps=59.94,
source="d")
67     flags = {f.code: f for f in rec.envelope.flags}
68     # frame-level evidence + fps attached so a CVAT (frame-based) reviewer can act
69     assert flags["overlap"].evidence.get("rally_frames") # rally 2's frame range
70     assert flags["overlap"].evidence.get("fps") == 59.94
71     # heuristic, review-only checks are framed gently (not as blockers) for the vendor
72     assert "Review only" in (flags["possible_missed_rally"].customer_message or "")
73
74
75 def test_delivery_clean_passes():
76     records = [_rec(1, 0.0, 5.0), _rec(2, 6.0, 9.0)]
77     rec = adapter.build_delivery_recorder(records=records, issues=[], fps=59.94,
source="clean")
78     assert rec.envelope.verdict == "pass"
79     assert _codes(rec.envelope) == set()
80
81
82 # --- import envelope -----
83 def _vmeta(fps=59.94, is_commercial=True):
84     return SimpleNamespace(
85         video_id="my_match", local_path="data/videos/my_match.mp4", fps=fps,
resolution="1080p",
86         license_type=SimpleNamespace(value="Proprietary"), is_commercial=is_commercial,
87         status=SimpleNamespace(value="curated"),
88     )
89
90
91 def test_import_clean_run():
92     anns = [SimpleNamespace(start_time=1.0, end_time=6.0),
SimpleNamespace(start_time=8.0, end_time=12.0)]
93     rec = adapter.build_import_recorder(video_meta_obj=_vmeta(), annotations=anns,
is_valid=True,

```

```

94             errors=[], count=2, sport="badminton",
source_format="csv")
95     env = rec.envelope
96     assert env.verdict == "pass"
97     assert env.get_stage("registration").details["is_commercial"] is True
98     assert env.highlights.segment_count == 2
99     assert env.lint() == []
100
101
102     def test_import_noncommercial_warns():
103         rec = adapter.build_import_recorder(video_meta_obj=_vmeta(is_commercial=False),
annotations=[],
104                                     is_valid=True, errors=[], count=0,
sport="badminton", source_format="json")
105         assert "license_noncommercial" in _codes(rec.envelope)
106
107
108     def test_import_missing_fps_warns():
109         rec = adapter.build_import_recorder(video_meta_obj=_vmeta(fps=None), annotations=[],
is_valid=True,
110                                     errors=[], count=0, sport="badminton",
source_format="csv")
111         assert "missing_fps" in _codes(rec.envelope)
112         assert rec.envelope.video_meta.fps_source == "unknown"
113
114
115     # --- emit (file writing) -----
116     def test_emit_delivery_writes_into_out_dir(tmp_path):
117         paths = emit_delivery_report(out_dir=str(tmp_path), source="pilot_01",
118                                     records=[_rec(1, 0.0, 5.0)], issues=[], fps=59.94)
119         assert paths is not None
120         assert (tmp_path / "pilot_01.report.json").exists()
121         assert (tmp_path / "pilot_01.report.md").exists()
122         assert (tmp_path / "pilot_01.summary.md").exists()
123
124
125     def test_emit_import_writes_next_to_video(tmp_path):
126         video = tmp_path / "clip.mp4"
127         video.write_bytes(b"x")
128         vm = SimpleNamespace(video_id="clip", local_path=str(video), fps=30.0,
resolution="720p",
129                             license_type=SimpleNamespace(value="MIT"), is_commercial=True,
130                             status=SimpleNamespace(value="curated"))
131         paths = emit_import_report(video_meta_obj=vm,
annotations=[SimpleNamespace(start_time=0.0, end_time=2.0)],
132                                     is_valid=True, errors=[], count=1, sport="badminton",
source_format="csv")
133         assert paths is not None
134         assert (tmp_path / "clip.report.md").exists()
135         assert (tmp_path / "clip.summary.md").exists()
136

```

28. assistant (2026-06-08T08:39:30.322Z)

Now let me check for any annotation metadata or tracking in the CVAT parser:

29. user (2026-06-08T08:39:31.148Z)

""Adapter: CVAT rally exports -> our canonical per-rally records.

Why this exists

Our annotation brief (`docs/annotation/R00RA_BRIEF.md`) asks vendors for a per-rally **CSV** that imports directly via `curate import-local`. A vendor may instead do the work in CVAT and hand back the **CVAT** export: per-frame bounding

boxes, with the rally fields (`rally\_number`, `shots\_count`, `ending\_reason` ...) carried as CVAT **object attributes** on each box. The box geometry is irrelevant to us ? it is just a carrier the rater attached the rally metadata to.

This module is the ingestion-side adapter that absorbs that vendor format so the modeling repo never has to. It does two things, both deliberately on *our* side so we fix mechanical problems rather than renegotiating them:

1. **Recompute time from the frame range.** A rally's interval is taken from the first/last *frame* a box for that rally appears on:
`seconds = frame / fps` at the video's true fps. We **ignore** any `start\_time`/`end\_time` attributes the vendor wrote ? a CVAT job that samples every `step` frames and assumes 30 fps yields systematically wrong times (we have seen `start\_time == frame / (step \* 30)`, i.e. off by the sampling step at the wrong fps). The frame index is reliable; the vendor's seconds are not.
2. **Surface ? never silently fix ? content defects.** Overlapping rallies, zero-duration rallies, ending_reason values outside the agreed set, and too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected rally boundary would poison ground truth, so that stays a human/vendor call.

Supported input: the single-file CVAT XML export, in either **image mode** (`<image><box>...`) or **track/interpolation mode** (`<track><box frame=...>`). The two PASCAL-VOC / COCO sibling exports encode the same data and can be added later if a vendor sends only those.

```
from __future__ import annotations

import os
import xml.etree.ElementTree as ET
from collections import Counter
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Tuple

from src.core.parsing import safe_int
from src.core.downloader import probe_resolution_fps
```

The agreed cross-sport ending_reason vocabulary for net-separated sports. Kept here as a soft check only (a warning, never a hard failure) so this adapter does not depend on the annotation-spec wording.

```
DEFAULT_ALLOWED_REASONS = {
    "winner",
    "forced_error",
    "unforced_error",
    "service_fault",
    "let",
    "other",
}
```

The observational outcome of the last shot ? orthogonal to ending_reason (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').

```
DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}
```

CVAT attribute names we map onto canonical columns. The vendor's CVAT label schema has used suffixed names like "score_before (A-B)"; we match leniently.

```
_ATTR_ALIASES = {
    "rally_number": ("rally_number",),
    "shots_count": ("shots_count",),
```

```

    "ending_reason": ("ending_reason",),
    "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome (in/net/out)"),
    "score_before": ("score_before", "score_before (a-b)"),
    "score_after": ("score_after", "score_after (a-b)"),
    "notes": ("notes", "note"),
}

CANONICAL_COLUMNS = [
    "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
    "shots_count", "ending_reason", "last_shot_outcome", "score_before", "score_after", "notes",
]

```

```

@dataclass
class RallyRecord:
    """One rally recovered from the CVAT export, time re-derived from frames."""
    rally_number: int
    start_time: float          # seconds, = frame_min / fps
    end_time: float           # seconds, = frame_max / fps (see padding note)
    shots_count: Optional[int]
    ending_reason: Optional[str]
    last_shot_outcome: Optional[str]
    score_before: Optional[str]
    score_after: Optional[str]
    notes: Optional[str]
    frame_min: int
    frame_max: int
    n_boxes: int
    vendor_start_time: Optional[str] = None    # the (discarded) attribute value
    vendor_end_time: Optional[str] = None

    @property
    def duration(self) -> float:

```

30. assistant (2026-06-08T08:39:33.076Z)

Let me look at the Issue class in the parser:

31. user (2026-06-08T08:39:33.746Z)

```

class Issue:
    """A defect found in the delivery. ``blocker`` issues stop a clean import."""
    severity: str    # 'blocker' | 'high' | 'medium' | 'low'
    code: str
    message: str
    rally_number: Optional[int] = None

```

```

@dataclass
class ConversionResult:
    records: List[RallyRecord]
    issues: List[Issue]
    fps: float
    step: int
    meta: Dict[str, Any] = field(default_factory=dict)

    @property
    def n_blockers(self) -> int:
        return sum(1 for i in self.issues if i.severity == "blocker")

```

32. assistant (2026-06-08T08:39:36.447Z)

Perfect. Now let me create a comprehensive summary based on all the files I've read:

33. user (2026-06-08T08:40:02.114Z)

Structured output provided successfully

34. assistant (2026-06-08T08:40:11.494Z)

Summary

I've completed a thorough read of the collector DB/provenance and observability layer. Here are the key findings:

Annotation Registration & Provenance

****How ingested annotation sets are registered with provenance:****

- `VideoMetadata` table stores: `video\_id`, `sport`, `license\_type` (enum), `license\_url`, `is\_commercial` (boolean flag), `status` (STAGING/CURATED/REJECTED), `match\_name`, `fps`, `resolution`
- License types include Proprietary (self-rated) and vendor-delivered types (MIT, Apache-2.0, CC-BY, etc.), but ****no source marker**** distinguishes them
- `is\_commercial` is derived from a hardcoded whitelist that includes `Proprietary`, treating self-rated licenses as automatically commercial-safe

****Status/Maturity fields:****

- Only ****video-level**** curation status exists (STAGING/CURATED/REJECTED)
- ****Segments lack maturity states**** ? rallies/points/deliveries have no individual review status (no needs_review ? confirmed ? gold tracking)
- Local imports skip staging: `import\_local` hardcodes CURATED status, bypassing vendor review workflow

Import Observability & Reporting

****Import observability envelope:****

- Emitted via `emit\_import\_report()` (soft dependency on obsreport)
- Captures: `parse` stage (records_parsed), `validation` stage (is_valid boolean, errors as strings), `registration` stage (sport, license, is_commercial, status)
- ****No segment-level flags**** ? import validation reports only temporal integrity (overlaps, duration), not QA metrics
- ****Asymmetry with delivery validation:**** CVAT delivery parser emits structured `Issue` objects (blocker/high/medium/low), but `import\_local` does not surface Issues

License & Provenance Tracking

- ****Proprietary self-rated**** and ****vendor-delivered**** licenses are NOT distinguished in schema
- License commerciality is checked against a config whitelist (defaults to [MIT, Apache-2.0, BSD, CC0, CC-BY, Public-Domain, ****Proprietary****])
- Observability spec rule `license\_noncommercial` warns only if `is\_commercial==false`; no audit trail for why Proprietary is safe

Critical Gaps

1. ****No annotator/author metadata**** ? neither video nor segment schemas capture creator/reviewer or timestamps
2. ****No segment maturity tracking**** ? can't mark labels as needs_review, confirmed, or gold
3. ****No re-tagging audit trail**** ? `insert\_badminton\_rallies()` DELETE-then-INSERTs, erasing history
4. ****No label quality regression detection**** ? no comparison metrics when re-importing annotations
5. ****Observability rules are video-only**** ? spec.yaml defines no segment-level rules (gaps, outliers, schema violations)
6. ****Import validation is weak**** ? only checks temporal integrity, not vocabulary/semantic validity like CVAT Issues do
7. ****License source opacity**** ? Proprietary tag doesn't indicate if self-rated (local expert) vs. vendor-rated (external source)